

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题 目： 多核神经网络处理器的计算图切分与调度优化

摘 要：

神经网络处理器（neural processing unit, NPU）包含多个计算核心（NPU core，图中简称 Core）。待处理的计算任务包含一组运算、数据搬运以及它们的先后关系。将操作和数据分别表示为节点，用有向边表示操作读取或生成哪些数据，就得到本题的计算图。本文把图中的计算操作划分为若干组，连同相关数据和依赖构成原图的子图；各核心实际执行的是这些子图所对应的计算任务。因此，需要共同决定操作怎样分组、各组任务交给哪个核心，以及任务按什么顺序执行。

问题一规定，每个子图对应一个独立下发的执行任务（Task），任务之间传递的数据必须经过外部主存。算法寻找互不需要对方结果的两组运算。在满足后文给出的任务高度、接收核心任务顺序及外部依赖条件时，将其中一组移交另一核心，并把需要读取两组结果的后续运算单独安排。任务顺序若相互冲突，可能导致各任务都在等待尚未完成的其他任务。因此，本文先证明所给调整条件能够避免这种循环等待，再比较调整前后的总体任务执行时间（Makespan）。

问题二把分配到同一个核心的各子图所对应的操作合并到一个 Task 中，允许后续操作直接使用该核心缓存中保留的数据。算法同时考虑生成、读取同一数据的操作应放在哪些核心，以及计算单元何时空闲：前者用于计算复制字节数，后者用于安排计算位置与次序。问题三进一步增加各核心共享的只读缓存。算法调整计算顺序，以改变结果需要保留的时间和后续读取的时刻；对于中间结果只被唯一父操作读取的树状计算，证明在逐棵子树完整处理的限定模型下，所给排序使内部结果的最大存储量最小。

三个固定算法版本均完成 100 张官方计算图、每图 1~5 个核心的评价，每题得到 500 个合法结果。问题一在 2~5 个核心上的平均加速比为 1.9516、2.7514、3.4786、4.0553；问题二为 2.3167、3.2356、3.9711、4.5498。问题三在 5 个核心上的平均基线加速比为 4.7576；保持调度方案不变，启用只读缓存相对关闭缓存的逐图平均加速比为 1.0829，逐图字节命中率均值为 31.36%。问题一 5 个核心时的平均求解时间为 3.522 秒，问题二全部测试的平均求解时间为 4.029 秒，均包含生成和比较方案所需的时间。

实验中也出现复制量减少但完成时间变长的方案。因此，本文先用依赖和存储关系提出调整，再按官方程序对完整方案的模拟结果选择输出；复制字节数和局部存储量只承担其定义范围内的判断，不代替最终完成时间。

关键词： 多核调度；计算图切分；数据搬运；缓存复用；总体任务执行时间

1 问题重述与分析

1.1 研究对象与优化目标

本题面向神经网络处理器（neural processing unit, NPU）的多核调度优化问题。NPU 包含多个计算核心（NPU core, 下文正文统称为“核心”，图表中简称为 Core）。NPU 代表整颗处理器芯片系统，而核心是指其内部并行的独立计算与控制单元。每个核心配备专用的矩阵计算单元 Cube、向量计算单元 Vector、核内核外数据搬运单元，以及该核心独占访问的私有片上缓存——局部存储器 L1 与统一缓冲区（Unified Buffer, 简称 UB）。各核心之间无法直接相互读写对方的私有存储空间，所有核心通过总线共享核外主存储器，题面简称为 DDR（源自双倍数据率动态随机存取内存，double data rate dynamic random-access memory）。DDR 的总读写带宽、片上私有缓存容量以及各类硬件操作之间的同步等待周期均由硬件系统固定。

交由 NPU 执行的计算任务，在底层由一系列具有前后依赖关系的算子运算及其读写的数据组成。一个算子通常需要等待其前驱算子输出所需的数据后方可启动。为了形式化地刻画这些运算任务及时序与数据流动约束，题面将待执行的神经网络模型表示为**计算图（computational graph）**。算法的目标是在给定的硬件约束下，共同决定计算任务的操作划分、核心分配以及执行次序 [1]。

以两次相互独立的矩阵乘计算为例，将它们分别分配至两个核心可以实现时间上的并发执行；然而，若两项计算均需从核外 DDR 载入大规模输入张量，它们就会在总带宽有限的 DDR 访问通道上产生争用。此外，若后续运算必须等待这两次矩阵乘全部结束方能汇总，系统还必须计入数据传输延迟与跨核心同步等待。由此可见，单纯平衡各核心的计算量只是调度优化的基础前提，完成时间的长短更取决于数据搬运开销、共享带宽争用以及依赖等待链的综合作用。

本题的首要评价指标为总体任务执行时间（Makespan），即在官方离散事件模拟评估程序下，从所有操作的最早启动时刻到全部操作均完成的最晚结束时刻所经历的硬件时钟周期数（cycle）。与方案质量指标相对应的是算法本身的求解效率，即算法程序在通用计算设备上从读取输入图到持久化输出合法方案所耗费的物理墙钟时间（Wall-clock Time），本文统称为求解时间（以秒为单位）。前者衡量调度方案在模拟环境下的执行性能，后者反映算法的实用工程效率。本文报告的执行时刻与周期数均源自官方评测系统的模拟推进，不作为物理硬件测量的绝对性能指标。

1.2 计算图的数据表示与数据依赖

题面将神经网络抽象为二部有向无环图（bipartite directed acyclic graph, bipartite DAG）形式的计算图。图中的节点集合被严格划分为两类互不相交的顶点：算子节点（Op node, 表示一次具体的数学计算或数据搬运）与数据张量节点（Tensor node, 表示输入数据、计算中间结果或最终输出数据）。

有向边仅连接不同类别的节点，表达数据的生成与消费逻辑：若算子 u 运算生成数据张量 t ，则计算图中存在有向边 $u \rightarrow t$ ；若算子 v 需要读取张量 t 作为计算输入，则存在有向边 $t \rightarrow v$ 。图中不存在算子到算子或张量到张量的直接连边。张量节点包含 **size** 属性，记录该数据块所占用的物理存储大小（以字节 Byte 为单位）。

根据计算图的有向拓扑，操作之间存在两类依赖关系：若算子 u 生成张量 t ，且算子 v 直接读取该张量 t （即二部图中仅通过单一中间张量节点连接 $u \rightarrow t \rightarrow v$ ，且不经任何其他计算操作），则称操作 v **直接数据依赖** 于操作 u ；此时，后继操作 v 的开始时刻不得早于前驱操作 u 的完全完成时刻。若从操作 u 到操作 v 的有向路径中途经过了一个或多个其他中间操作（即形成了 $u \rightarrow \dots \rightarrow w \rightarrow \dots \rightarrow v$ 的传递链条），则称操作 v **间接数据依赖** 于操作 u 。只要两个操作之间存在直接或间接数据依赖，后继操作 v 的开始时刻就必须滞后于前驱操作 u 的结束时刻。若两个计算操作仅共同读取同一份外部输入张量，仅就该输

入共享关系本身而言并不构成先后依赖；但二者仍可能在计算图中通过其他路径存在先后数据依赖，需结合全图拓扑综合判定。计算图的有向无环特性确保了计算流单向推进，不存在自相矛盾的时序闭环。

在后续算法设计与拓扑分析中，为了清晰展示计算任务的先后偏序关系，我们常将二部图向仅包含算子节点的**算子依赖图**进行投影：对二部图中每一个张量节点 t ，若算子 u 是生成 t 的操作 ($u \rightarrow t$)，且操作集合 $\{v \mid t \rightarrow v\}$ 读取 t ，则在算子依赖图中由 u 向该集合中的每一个读取算子 v 引出一条有向依赖边 $u \rightarrow v$ 。算子依赖图是用于简化依赖推导的辅助表示，底层的存储占用、搬运开销以及缓存判定仍基于带有张量大小属性的原始输入图展开。

根据官方输入输出契约，题面预设了两类核外数据搬运操作：**COPY_IN** 负责将指定张量从核外 DDR 搬运至对应核心的私有缓存（L1 或 UB）；**COPY_OUT** 负责将计算核心内部私有缓存中的数据写回至 DDR。算子的唯一标识编号（Op ID）与张量的标识编号（Tensor ID）属于两个独立的命名空间，分别唯一确定底层的调度单元与数据对象。

1.3 执行资源与计算和数据搬运的时间重叠

在每个核心内部，物理硬件被组织为若干专用的硬件执行管道（Pipe），题面将其运行机制统称为**硬件流水线（pipeline）**。在本文所针对的 NPU 体系结构语境下，硬件流水线（pipeline）特指核心内部服务于特定指令类型的微架构执行资源及其严格保序的处理机制。根据题面硬件建模，每个核心包含四条相互平行的 Pipe：

1. **PIPE_M**：驱动矩阵计算单元 Cube，专门负责矩阵乘累加等密集矩阵算子；
2. **PIPE_V**：驱动向量计算单元 Vector，专门负责向量算术、非线性激活等向量算子；
3. **PIPE_MTE2**：驱动核内数据搬运单元，负责将数据从核外 DDR 搬入私有缓存 L1/UB，或在私有缓存内将数据从 UB 转移至 L1；
4. **PIPE_MTE3**：驱动核内数据搬运单元，负责将私有缓存 L1/UB 中的数据写回至 DDR，或在私有缓存内从 L1 转移至 UB。

官方离散事件评测程序对流水线推进作出了明确的时序约束：在单个核心内部，同一条 Pipe 在任意时刻至多只能执行一个操作；不同 Pipe 之间只要满足数据依赖和片上存储容量条件，则允许在时间轴上并发推进。

例如，当核心内的向量流水线（PIPE_V）正在对已位于私有缓存中的数据执行运算时，搬运流水线（PIPE_MTE2）可以同时从核外 DDR 载入后续算子所需的其他输入张量。两项操作如果在时间轴上拥有一段共同的执行区间，便在逻辑上达成了**计算与数据搬运的时间重叠（overlap of computation and data transfer）**。在模拟模型中，实现时间重叠能够有效减少处理器的空转等待。能否顺利实现时间重叠，取决于输入数据是否已传输到位、对应执行 Pipe 是否处于空闲状态、所属任务是否获准激活，以及私有缓存 L1/UB 中是否有足够的可用存储容量以容纳输出张量。若因等待前驱结果或缓存不足导致 Pipe 空闲停顿，系统时间线上就会产生调度气泡（Bubble）。

1.4 计算操作的分组、分配与调度方案表示

面向多核心体系架构，算法需要协同确定三项核心决策：

1. **操作划分**：将计算图中全部非核外搬运的待调度算子划分为若干互不重叠的集合，每个集合连同其引用的局部张量及依赖边构成原计算图的一个**子图（subgraph）**；
2. **核心分配**：将划分出的各个子图分别映射指派至特定的 NPU 核心负责处理；
3. **时序编排**：在同一个核心内部，确定该核心所承载的各个子图之间的执行先后次序。

根据官方提交契约（contest_io.py），提交方案必须且仅包含 `node_to_subgraph` 和 `core_schedules` 两个字段。其中 `node_to_subgraph` 映射字典负责记录原计算图中操

作类型不属于 COPY_IN 与 COPY_OUT 的全部算子所属的子图编号（核内搬运或其它非核外 COPY 算子仍须纳入划分）；core_schedules 列表为每个核心分别给出该核心所分配子图编号的执行顺序序列。方案必须保证上述全部待调度算子被无遗漏、无重复地覆盖，且每个子图编号在所有核心的调度列表中恰好出现一次。

在评测系统的执行语义中，**执行任务（Task）**是分配给特定核心执行的计算单元。子图是算法在计算图拓扑层面划分的操作集合，而 Task 则是评测模型运行时的调度与隔离边界，核心实际执行的是子图所对应的计算任务。赛题所提出的三个问题，其本质差异正是由子图到 Task 的组织关系以及片上数据驻留规则的改变所界定的：

机制与规则维度	问题一	问题二	问题三
Task 的组织构成	每个子图独立构成一个 Task	同一核心所分配的全部子图合并为一个统一的 Task	与问题二相同
同核跨子图数据保留	严禁保留，必须经由 DDR 写回与重新载入	允许在私有缓存 L1/UB 中跨子图直接驻留与复用	与问题二相同
同一核心内切换延迟	连续 Task 间必须等待 100 cycles	核心仅执行单一合并 Task，无内部 Task 切换延迟	与问题二相同
跨核心通信同步延迟	跨核前驱 Task 完成后需等待 1000 cycles	对应源 COPY_OUT 完成后需等待 500 cycles	与问题二相同，即使命中只读 Cache 也需满足该时序约束
共享二级缓存（L2）	无，核心仅能访问私有 L1/UB 与共享 DDR	无，核心仅能访问私有 L1/UB 与共享 DDR	引入核外共享只读缓存（Level-2 Cache，简称 L2）

由上述规则演进可知：

- 问题一将每个子图视为隔离的 Task，算法面临增加多核并行度与增加 DDR 搬运及 Task 切换边界之间的直接冲突；
- 问题二通过在同一核心内将子图合并为大 Task，解除了同核跨子图的数据强制落盘限制，允许中间结果在私有片上缓存中直接驻留复用，但中间张量在缓存中停留越久，对私有存储空间的占用压力就越大；
- 问题三在多核合并执行的基础上引入了全局只读 Cache，多个核心读取相同张量时，若数据已驻留于 Cache 则产生**缓存命中（cache hit）**，可直接从高速 Cache 通道读取而无需重复向 DDR 发起请求，从而有效分流共享带宽；若不在 Cache 中则发生**缓存未命中（cache miss）**，需从 DDR 搬入并在搬入完成后按先进先出（FIFO）规则写入 Cache；单张量大小超过 Cache 总容量时则不予缓存。

图 1.1 针对相同的代表性计算片段，直观对比了三类问题在 Task 构成、数据流动路径以及多核心交互方式上的结构演变。

1.5 多核调度中的核心挑战与求解思路

综合题意与硬件执行特征，在通用神经网络处理器上实现高效的多核调度存在以下三项核心难点：

第一，**并行计算收益与跨核通信开销之间的非线性权衡**。将计算图拆解为更多子图并分散至更多核心，理论上能够提升计算单元的宏观并发度；然而，跨核心的数据交换必须插入额外的 COPY_OUT 与 COPY_IN 搬运操作。所有核心的 DDR 搬运操作均受制于总带宽仅为 60 字节/周期的共享内存通道，密集的跨核通信极易引发严重的内存带宽争用与同步

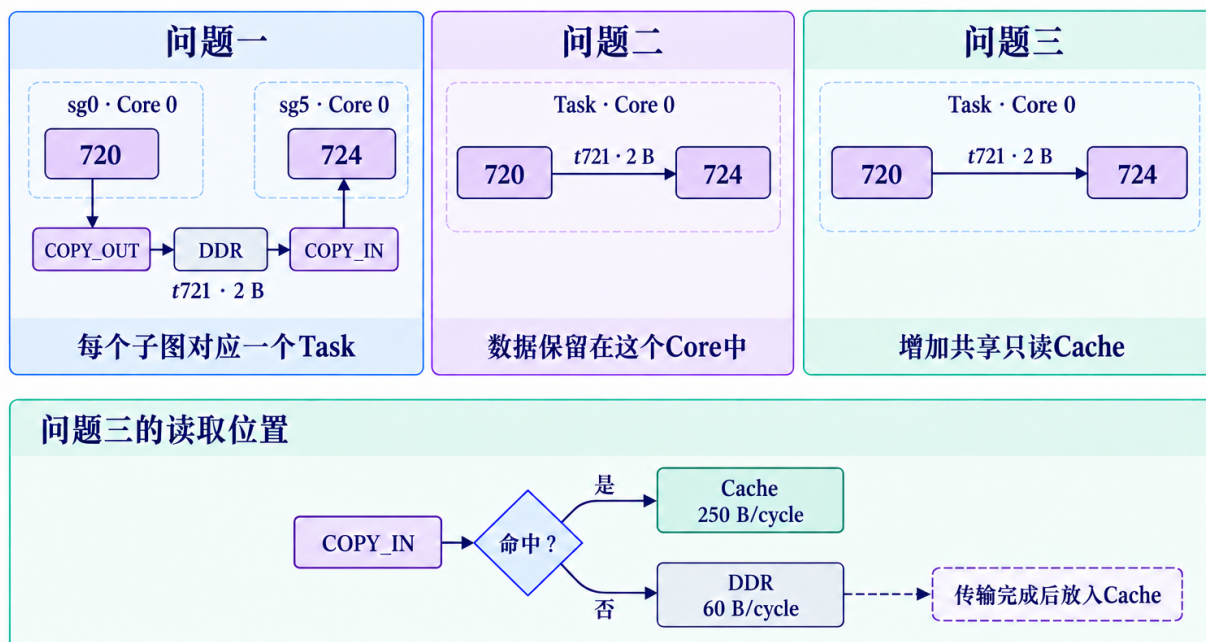


图 1.1 三问条件的差别。问题一为图中不同子图所对应的操作分别建立 Task，数据经 DDR 中转；问题二允许同一个核心保留数据；问题三另为符合条件的 COPY_IN 提供共享只读 Cache。下部示意数据能够装入 Cache 时的读取过程；所有请求仍须满足原依赖及规定等待。

停顿。如果因通信导致的额外 DDR 搬运与等待所增加的执行周期，超过了并行分配所节省的计算周期，多核方案的总完成时间反而会恶化。

第二，**片上私有缓存容量限制与数据驻留的动态约束**。在问题二和问题三中，将具有数据依赖的操作集中至同一核心能够避免外部内存搬运；然而，核心内部 L1 和 UB 的私有存储容量是严格受限的。核内调度在编译阶段若检测到局部缓存容量不足，会预先插入将数据换出至核外 DDR、后续再换入的溢出搬运操作（spill）。这不仅会增加额外的 DDR 搬运开销，若调度依赖安排不当还会导致方案无法推进。

第三，**局部核心排序与全局数据依赖交织下的死锁风险**。通过反例，我们可以说明为什么仅仅保证原始计算图无环并不能确保多核调度方案合法：假设核心 0 上的子图任务 S_1 需等待核心 1 上的子图任务 S_2 提供数据，而核心 1 内部指定的调度次序又要求先执行任务 S_3 再执行任务 S_2 ；若任务 S_3 恰好又间接依赖于任务 S_1 的运算结果，两个核心便会陷入“ S_1 等待 S_2 、 S_2 等待 S_3 、 S_3 等待 S_1 ”的循环依赖状态，导致评测系统发生死锁。

针对上述挑战，本文遵循题面关于避免暴力迭代搜索、单用例高效求解的指导原则，设计了结构化的求解流程：

1. **基于图结构的特征分析与候选构造**：分析计算图的连通分支、关键路径与张量复用关系，针对独立并行分支与数据密集型计算链，直接构造少量具有结构特征的完整候选方案；

2. **调度顺序的无环性判定**: 将核心内子图执行次序作为时序边显式引入子图依赖图中, 证明所给次序不产生有向环, 确保方案满足依赖先后关系 (需要说明的是, 顺序无环是合法执行的必要条件, 但仍需结合存储容量与完整评测进一步检验);
3. **结合通信代价模型的理论下界快速剪枝**: 基于题面给定的硬件流水线资源与通信带宽约束, 计算理想并发条件下的时间下界与跨核通信开销, 辅助快速筛除劣质候选;
4. **离散事件模拟选优**: 在受限的评估预算内, 调用评测程序对生成的完整候选方案进行离散事件模拟, 从中选取 Makespan 最小的方案作为输出。

2 模型假设与符号说明

2.1 硬件参数与执行约束

本文的建模与算法设计严格建立在题面给定的体系结构参数与官方离散事件评测程序的基础上。计算图中各算子的计算语义、操作类型及原始数据依赖保持不变, 算法不得通过删减计算或修改运算结果来缩短完成时间。全部存储容量、总线带宽及同步等待常数均采用官方固定配置 [1]。

根据硬件配置参数, 系统的主要物理约束如下:

1. **核心数量**: 可用核心数 $K \in \{1, 2, 3, 4, 5\}$, 系统为各核心编号为 $0, 1, \dots, K - 1$;
2. **片上私有缓存**: 每个核心配备独立的局部存储器 L1 与统一缓冲区 UB, 其存储容量分别为 524,288 B (512 KiB) 与 131,072 B (128 KiB)。两级缓存各自独立管理, 未分配的 L1 容量无法与 UB 共享;
3. **共享主存与缓存带宽**: 所有核心访问核外主存 (DDR) 的读写操作共享统一的全局带宽池, 总带宽固定为 60 B/cycle。在问题三中, 引入容量为 1,048,576 B (1 MiB) 的共享只读二级缓存 (Level-2 Cache, 简称 L2), 命中 Cache 的读取操作使用专用的共享 Cache 读取带宽通道, 总带宽为 250 B/cycle, 该带宽与 DDR 带宽相互独立, 不占用 DDR 带宽池;
4. **同步与切换延迟**: 在问题一中, 同一核心连续执行的相邻 Task 之间须等待 100 cycles 的切换时间; 若一个 Task 依赖位于其他核心的前驱 Task, 其启动时刻还须满足晚于前驱 Task 完成时刻至少 1000 cycles 的跨核同步延迟。在问题二和问题三中, 由于同核子图合并为单一 Task, 核心内部不存在 Task 切换延迟; 跨核心依赖仅作用于具体数据搬运操作, 目标核心的 COPY_IN 需在源核心对应的 COPY_OUT 完成后等待 500 cycles 方可参与就绪判断。这些常数由评测程序在对应的事件发生时检查, 不直接等同于图上的固定边权重。

评测程序采用离散事件模拟推进执行。对于流水线操作队列, 算子严格按核内静态调度生成的偏序队列以先进先出 (first in, first out, FIFO) 方式推进就绪与发射。对于问题三中的只读 Cache, 其缓存行维护独立的 FIFO 淘汰队列: 当 Cache 剩余容量不足以容纳新载入的张量时, 系统依次淘汰最早写入 Cache 的条目; 后续核心读取 Cache 发生命中时, 只读访问不会更新该条目在 FIFO 队列中的淘汰顺序。单张量物理大小超过 Cache 总容量 (1 MiB) 的数据不进入 Cache。

若核内调度算法在静态编译时评估到私有片上缓存 (L1 或 UB) 容量不足以容纳计算过程中的所有活跃张量, 评测程序会在该核心的指令流中预先插入溢出搬运操作 (spill), 即通过 PIPE_MTE3 将暂不使用的中间数据经由 DDR 写出, 后续需要时再通过 PIPE_MTE2 重新从 DDR 读入。溢出搬运会产生额外的 DDR 访问流量并占用共享带宽, 调度方案应尽量平抑片上张量驻留峰值以减少溢出开销。

2.2 原始张量、数据副本与逻辑标识

在计算图的生命周期中，同一份数学数据在不同调度阶段可能对应不同的图节点与存储实体，我们明确区分以下三类对象：

1. **原图张量节点**：指原始输入计算图 JSON 文件中给定的张量实体，具有唯一的整型 `id` 和固定的数据大小 `size`；
2. **调度生成的数据副本**：在多核心切分与核内调度阶段，当数据发生跨核心传输或因容量受限触发溢出时，评测程序会为该数据在不同存储介质（如源核心私有缓存、核外 DDR、目标核心私有缓存）中派生出新的物理数据实例。这些副本虽然承载相同的内容，但在物理上分别占用各自存储介质的独立空间；
3. **逻辑数据标识 (`logical_tid`)**：评测系统在处理问题三的只读 Cache 查询时，为所有源于同一张量派生出的物理副本赋予统一的逻辑标识 `logical_tid`。当核心发起 `COPY_IN` 搬运时，评测程序依据请求数据的 `logical_tid` 查询 Cache 队列。若命中则直接从 Cache 读取，若未命中则在 DDR 读取完成后以该 `logical_tid` 登记至 Cache 中。

此外，方案合法性必须满足三重判定条件：

- **操作覆盖合法**：输入图中除 `COPY_IN` 与 `COPY_OUT` 之外的全部算子，均被无重复、无遗漏地指派至唯一的子图，且每个子图在 `core_schedules` 中恰好出现一次；
- **依赖拓扑无环**：子图间的数据依赖与各核心指定的子图执行顺序合并构成的扩展依赖图不存在有向环；
- **存储与执行可行**：评测程序对方案能够成功完成核内调度并推进离散事件模拟，不因容量冲突或死锁导致模拟异常终止。

只有评测程序完整运行成功并输出日志的方案，其导出的 `Makespan` 才是有效指标。若方案因违反依赖或容量超限被评测程序拒绝，或者执行发生超时，均视为无效解，不将其填为零参与均值统计。

2.3 主要符号

本文使用的主要数学符号定义如下表所示。

符号	定义说明	取值范围或量纲
$G = (O \cup T, E)$	原始计算图，其中 O 为操作节点集合， T 为张量节点集合， E 为有向依赖边集合	题面输入给定
V	待调度的计算算子集合（即 O 中操作类型不为 <code>COPY_IN</code> 和 <code>COPY_OUT</code> 的算子）	$V \subset O$
b_t, ℓ_t	数据张量 t 的物理大小（字节）及其预设存储位置（DDR、L1 或 UB）	$b_t \in \mathbb{N}^+ (\text{B})$, $\ell_t \in \{\text{DDR}, \text{L1}, \text{UB}\}$
d_v, p_v	算子 v 的硬件执行周期数及其所绑定的硬件执行管道 <code>Pipe</code>	$d_v \geq 1 (\text{cycle})$, $p_v \in \{\text{PIPE_M}, \text{PIPE_V}, \dots\}$
K	系统可用的计算核心总数	$K \in \{1, 2, 3, 4, 5\}$

符号	定义说明	取值范围或量纲
$\pi(v)$	算子划分映射，表示算子 v 所归属的子图编号	$\pi : V \rightarrow \mathbb{N}$
$a(s)$	子图分配映射，表示子图 s 所指派执行的核心编号	$a : \mathcal{S} \rightarrow \{0, 1, \dots, K - 1\}$
σ_k	核心 k 所承担的子图编号的有序调度列表	子图编号的全排列序列
$P = (\pi, a, \sigma)$	完整的多核切分与调度方案 (对应提交 JSON 文件的两个字段)	参赛提交规范
q	赛题问题编号	$q \in \{1, 2, 3\}$
$M_q(P)$	方案 P 在问题 q 的评测环境下运行所得的总体任务执行时间 (Makespan)	cycle
$D_q(P)$	方案 P 在评测程序下产生的总额外 DDR 数据搬运量	B (字节)
B_i	第 i 张计算图在官方基准单核心调度算法下的基线 Makespan	cycle
$H_{i,K}$	第 i 张计算图在 K 个核心下运行问题三方案所得的 Cache 字节命中率	$[0, 1]$
L, U	在相同图输入、核心数与硬件配置下，对所有合法方案均成立的理论下界与已有合法方案的时间上界	cycle
$L(P)$	仅在指定候选方案 P 的特定结构约束下成立的时间下界 (不作为全局下界)	cycle
U/L	当前最佳方案的 Makespan 与理论下界之比，用于评估当前方案逼近理论极限的程度	无量纲比值
T_{solve}	求解算法从读取输入到输出合法方案文件的端到端墙钟运行时间	s (秒)

在官方输入文件中，非核外搬运计算算子 $v \in V$ 的 `cycles` 字段可能为 0。根据官方评测程序逻辑，计算单元在单核心执行推进时确保每个计算算子至少占用 1 个执行时钟周期，因此在建立数学分析模型与计算计算负载下界时，算子执行周期定义为 $d_v = \max(1, \text{cycles}_v)$ (单位为 cycle)。需要特别说明的是， d_v 仅适用于具备固定计算时长的算子集合 V ；对于核外数据搬运操作 (COPY_IN 与 COPY_OUT)，其执行耗时由张量大小 b_t 与实时动态共享带宽池决定，不适用该静态公式。

2.4 评价指标及统计方法

2.4.1 1. 方案质量指标与平均加速比

多核心调度方案的质量首要由 Makespan $M_q(P)$ 衡量。为了评估多核并行化相对于单核执行的性能提升程度，根据题面第 5 页脚注 2 的明确规定，多核心调度方案的平均加速比（Speedup）采用逐用例比值的算术平均方法进行计算：

$$\bar{S}_{q,K} = \frac{1}{100} \sum_{i=1}^{100} \frac{B_i}{M_q(P_{i,K})}, \quad (2.1)$$

其中 B_i 为官方评测系统针对第 i 张图在单核心条件下的基线 Makespan， $P_{i,K}$ 为算法针对第 i 张图在 K 个核心下生成的调度方案。必须说明的是，平均加速比是逐图加速比值的算术平均值，而非全部用例单核时间总和除以多核时间总和。题面官方基线 B_i 由通用的单核启发式调度算法给出，并不保证是单核条件下的数学全局最优解，因此在部分用例中可能出现加速比略大于核心数 K 的情况。

对于问题三，题面要求评估引入只读 Cache 带来的相对性能增益。评测时保持问题三所生成的调度方案 P 完全不变，分别在启用 Cache 和关闭 Cache（即回退至问题二纯 DDR 模式）两种配置下运行评测程序，分别记录其 Makespan 为 $M_3(P)$ 和 $M_2(P)$ 。单用例的 Cache 加速比定义为 $M_2(P)/M_3(P)$ ，多用例汇总时同样报告这 100 对结果比值的算术平均值。

2.4.2 2. 额外数据搬运量与缓存命中率

次要方案质量指标包括数据搬运量与缓存命中率：

- **总额外 DDR 数据搬运量 $D_q(P)$** ：官方评测程序统计调度后执行的所有 COPY_IN 和 COPY_OUT 搬运字节总数，减去原始计算图固有输入输出张量所需的基准搬运字节数。在问题三中，官方评测系统保持相同的统计口径，并不在 Cache 命中时将该统计字段直接扣减，因此该字段反映的是切分引入的总通信需求，而非物理 DDR 总线的实际净流量；
- **Cache 字节命中率 H** ：定义为命中 Cache 的数据总字节数占有所有发起 Cache 查询的总数据字节数的比例，即

$$H = \frac{\text{hit_bytes}}{\text{hit_bytes} + \text{miss_bytes}}. \quad (2.2)$$

当用例中不存在任何可查询的 COPY_IN 操作时，官方程序将分母为零的情形约定为 $H = 0$ 。该指标严格按字节量统计，而非按访问请求的发生次数统计。

2.4.3 3. 求解效率与原题时限解读

算法的求解效率通过端到端求解时间 T_{solve} 评估，度量从程序启动、解析输入图 JSON，到完成拓扑分析、方案构造、内部筛选及输出方案文件全过程的实际物理墙钟耗时（Wall-clock Time）。

原题第 5 页脚注 1 指出：“此处‘高效地’代表着算法应该避免使用暴力迭代搜索机制，以单个用例能在 5~10 分钟内给出结果为佳。”我们对该项要求的理解为：

- 该项表述属于针对求解算法工程效率与设计理念的指导性建议，强调算法应当利用计算图的固有结构与硬件特性进行针对性构造，避免盲目的大规模蒙特卡洛枚举或无剪枝的暴力迭代；
- 5~10 分钟是算法在常规计算硬件上的用例级运行耗时建议，并非超过 600 秒即被判定无效的硬性淘汰线，亦非要求算法必须耗满 5 分钟方可输出。本队算法在保证方案高质量的前提下，追求极高的求解速度，单用例求解耗时通常控制在数秒至数十秒之间。

2.4.4 4. 评测程序的角色与调用账目

在算法求解与评测交付中，明确区分在线候选评分（作为求解算法的一部分用于分支决策，计入求解墙钟时间）与外部官方复评（求解完成后独立运行以验证方案合法性并输出最终官方主成绩）：

- **E0（官方离散事件评测程序）**：官方提供的未修改基准评测脚本（`code/multicore_cut_evaluate_problem_*.py`）。在问题三求解过程中，算法基于归约树排序理论生成有限个候选方案后，直接调用 E0 进行在线完整打分验证（单用例调用预算上限为 3 次）；而在外部复评阶段，本文汇报的所有主实验成绩、最终 Makespan 以及附录逐用例结果，均以 E0 的独立运行输出为最终确认标准；
- **E1（批量精确评估工具）**：队内依据官方规则实现的轻量级独立评估实现。在问题一求解器中，用于对有限个候选方案（包括基础划分、局部调整及分支重分配方案）进行在线模拟打分与选优，单用例在线调用次数严格限制在至多 10 次以内；
- **E2（场景二轻量精确评估工具）**：队内针对问题二场景（同核子图合并为单一 Task、跨核以 COPY 依赖推进）实现的快速评估工具（在固定源码 `src/q2_nikolastarx/` 中由 `SceneBEvaluator` 提供支持）。在问题二求解器（`adaptive_hypergap_guarded.py`）中，算法基于超图多阶段划分与空隙插入构造出 `baseline`、`gap` 与 `hypergap` 等有限候选后，调用在线评分适配器进行打分（单用例在线调用上限为 3 次），并根据所返回的 Makespan 与额外数据搬运量自适应选取最佳方案；若 E2 接口调用异常则自动回退至 E0。

所有求解器内部的在线打分调用及其等待时间均严格计入算法的端到端求解时间 T_{solve} 中，未在求解阶段隐瞒在线评价开销。

3 计算图切分与调度模型

3.1 多核切分与调度决策的形式化表示

多核心调度算法的核心任务是在满足数据流依赖与硬件物理约束的前提下，共同确定算子到子图的分组、子图到计算核心的指派以及各核心内部子图任务的执行先后时序。

设 $V \subset O$ 为原始计算图中除核外搬运操作 `COPY_IN` 与 `COPY_OUT` 之外的全部待调度算子集合。方案的三个决策维度形式化定义如下：

1. **算子划分映射 π** ： $\pi : V \rightarrow \mathcal{S}$ 为每个算子 $v \in V$ 分配一个子图编号，所有非空子图组成的集合记为 $\mathcal{S}$ ；
2. **核心分配映射 a** ： $a : \mathcal{S} \rightarrow \{0, 1, \dots, K-1\}$ 为每个子图 $s \in \mathcal{S}$ 指定负责处理该子图计算任务的核心编号；
3. **核心内调度序列 σ_k** ：对于每个核心 $k \in \{0, 1, \dots, K-1\}$ ， σ_k 为指派给该核心的子图子集 $\{s \in \mathcal{S} \mid a(s) = k\}$ 给出一个严格的全序执行排列。

三项决策共同构成了完整的多核心切分与调度方案，记为三元组 $P = (\pi, a, \sigma)$ 。该数学表示与官方提交契约（`contest_io.py`）中的持久化结构完全对应：`node_to_subgraph` 字段直接对应映射 π ，`core_schedules` 字段直接对应核心执行序列集合 $\{\sigma_k\}_{k=0}^{K-1}$ 。单有算子分组而不指定执行核心，或者指定了分配而不确定同一核心上的执行次序，均无法在底层评测系统中推进离散事件时钟。

为了严谨刻画多核心场景下由数据依赖与核心内排队顺序交织构成的时序先后关系，我们建立子图层级的扩展有向拓扑：

- **子图间数据依赖边集 E_S** ：若子图 s 内存在算子生成的数据张量被子图 t 内的算子读取作为输入（ $s \neq t$ ），则在子图之间引入一条有向数据依赖边 $(s, t) \in E_S$ ；
- **核心内执行次序边集 E_σ** ：若在某个核心的执行序列 σ_k 中，子图 s 紧邻排定在子图 t 之前执行，则引入一条有向调度时序边 $(s, t) \in E_\sigma$ 。

将两类有向边与子图顶点集 $\mathcal{S}$ 合并，构成加入同一核心顺序后的子图依赖图：

$$D^+(P) = (\mathcal{S}, E_{\mathcal{S}} \cup E_{\sigma}). \quad (3.1)$$

若该有向图中存在有向环，则说明子图之间的数据流动与核心内的排队规则发生了循环互锁，方案在底层执行时必然引发死锁。

在方案满足拓扑无环性及格式规范的前提下，官方评测程序针对具体问题场景 $q \in \{1, 2, 3\}$ 执行离散事件模拟推进 $\mathcal{E}_q(P)$ ，生成方案在规定硬件环境下的性能评估向量 $(M_q(P), D_q(P), H_q(P))$ ，其中 $M_q(P)$ 为总体任务执行时间（Makespan，单位为 cycle）， $D_q(P)$ 为额外 DDR 数据搬运量（单位为 Byte）， $H_q(P)$ 为问题三特有的 Cache 字节命中率。

设 $\mathcal{F}_q(G, K)$ 为评测程序能够顺利推进并正常完成的所有合法方案集合，多核心调度问题的数学优化模型表述为：

$$\min_{P \in \mathcal{F}_q(G, K)} M_q(P). \quad (3.2)$$

在方案比选中，优化准则严格遵循题面要求：以最小化 Makespan $M_q(P)$ 为首要目标；当完成时间相同时，以减少额外搬运量 $D_q(P)$ 作为次要评价准则；对于问题三，仅当新方案的 Makespan 相对于无 Cache 模式或基线方案产生严格缩短时方予以采纳。

3.2 三个问题的通用求解策略与递进关系

求解该多核调度问题的根本难点在于：图论意义上的局部启发式指标（如划分割边权重、计算量均匀度）无法准确反映带有动态带宽争用与复杂同步等待的微架构执行过程；然而，若对搜索空间中的海量可能切分逐一运行完整的离散事件模拟评测，其时间开销在工程上又是不可接受的。

为了在合理的时间内产出高质量方案，本文设计了一套通用的结构化求解流程：

1. **图拓扑特征分析**：分析原始计算图的连通分支、长程关键路径以及张量复用聚集度，提取适合多核心展开的天然结构单元；
2. **结构化候选方案构造**：针对提取出的拓扑特征，直接生成少量包含切分、核心分配与执行顺序的完整候选方案 $P = (\pi, a, \sigma)$ ；
3. **拓扑合法性与无环性校验**：构建扩展依赖图 $D^+(P)$ ，校验所有待调度算子的无遗漏覆盖以及子图任务执行时序的无环性；
4. **理论下界剪枝**：计算针对具体候选方案的执行时间下界 $L(P)$ ，若其理论下界已劣于当前最佳方案，则提前剪除该候选；
5. **受限预算下的离散事件模拟选优**：对通过筛选的少量优质候选方案，调用离散事件评测程序获取真实性能指标，自适应输出 Makespan 最小的最优方案。

图 3.1 展示了该通用求解流程，三类问题的算法分支共享统一的输入解析、拓扑校验与闭环选优机制。

根据题面硬件执行语义的演变，三类问题的具体构造策略呈现清晰的递进逻辑：

- **问题一（子图隔离执行）**：每个子图独立封装为一个硬件 Task，子图间数据无法片上保留，通信代价高昂。算法侧重于识别无数据依赖的并行支路，通过分支重新分配均衡各核心的计算负载，平衡并行度与 DDR 通信开销；
- **问题二（同核子图合并）**：同一核心上的全部子图合并为单一 Task，允许片上私有缓存直接跨子图复用数据。算法基于超图划分理论，将具有密集张量复用的算子归并至同一核心，以最小化跨核心边界的数据通信，同时控制局部片上缓存峰值以规避溢出；

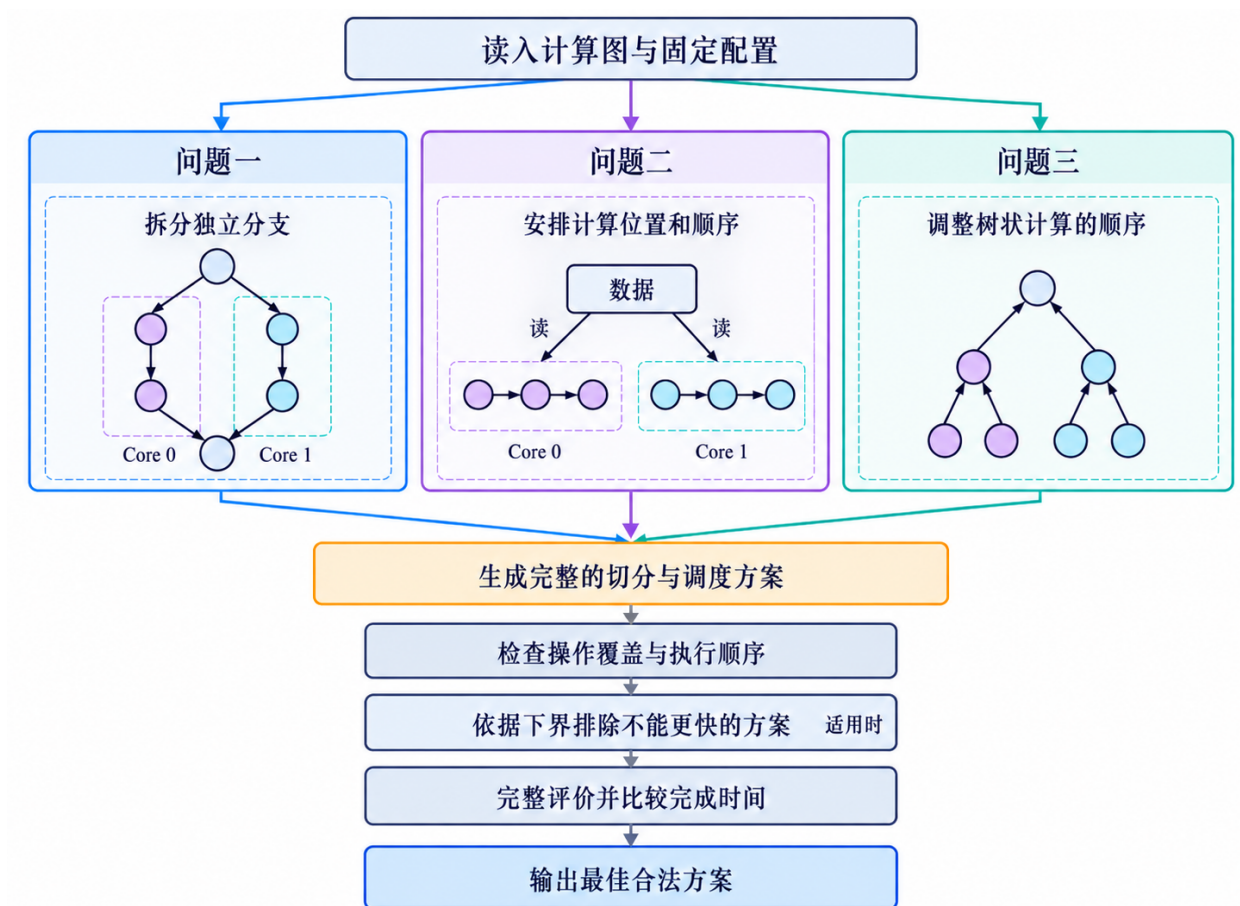


图 3.1 三问共用的方案构造与比较步骤。各分支只在结构条件满足时采用，所形成的完整方案需检查依赖与容量，并通过执行评价比较完成时间。下界只在其证明条件成立时用于提前排除候选。

- **问题三（引入共享只读 Cache）：**在问题二合并执行的基础上增加核外只读 L2 Cache。针对多核心读取共享数据的树状归约结构，算法基于深度优先排序理论排定子树计算次序，在非交错执行模型下最小化内存驻留峰值，最大化 Cache 复用收益。

3.3 子图划分与执行顺序的无环性判定

通过以下反例，我们可以说明为什么仅仅保证原始计算图无环以及每个子图内部无环，并不能确保多核心调度方案可安全执行。

假设原始计算图包含四个计算算子 a_1, a_2, b_1, b_2 ，它们之间仅存在两条数据依赖边 $a_1 \rightarrow b_1$ 与 $b_2 \rightarrow a_2$ 。显然，原始计算图是一个严格的有向无环图。若算法将算子划分为两个子图 $S_A = \{a_1, a_2\}$ 与 $S_B = \{b_1, b_2\}$ ，并分别分配至核心 0 与核心 1 执行。此时，子图间同时产生了由 S_A 到 S_B 的依赖（ $a_1 \rightarrow b_1$ ）以及由 S_B 到 S_A 的依赖（ $b_2 \rightarrow a_2$ ）。无论两个核心内部如何排定子图的执行次序，核心 0 上的任务与核心 1 上的任务都将陷入相互等待的死锁状态。不仅如此，即便子图间仅存在单向的数据传递，若核心内部所指定的调度序列引入了反向时序边，两类边叠加后仍可能形成环路。

因此，在对候选方案进行深入评价之前，必须形式化判定其扩展依赖图 $D^+(P)$ 的无环性。

命题 3.1 (无环性充分条件)。 设 $P = (\pi, a, \sigma)$ 为一份多核心切分与调度方案，其对应的扩展依赖图为 $D^+(P) = (\mathcal{S}, E_S \cup E_\sigma)$ 。若存在一个标号函数 $r : \mathcal{S} \rightarrow \mathbb{Z}$ ，使得对于图中任意有向边 $(s, t) \in E_S \cup E_\sigma$ ，均满足

$$r(s) < r(t), \quad (3.3)$$

则该扩展依赖图 $D^+(P)$ 必然不存在有向环，方案中的子图任务能够排定出合法的拓扑执行序列。

证明. 采用反证法。假设 $D^+(P)$ 中存在一个由 m 个子图构成的有向环 $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_m \rightarrow s_1$ (其中 $m \geq 2$)。根据命题假设，环上的每一条有向边 (s_i, s_{i+1}) 均满足严格的大小偏序关系：

$$r(s_1) < r(s_2) < \dots < r(s_m) < r(s_1). \quad (3.4)$$

由此直接导出 $r(s_1) < r(s_1)$ ，出现数学逻辑矛盾。因此，假设不成立， $D^+(P)$ 中必然不存在有向环。 $\square$

需要明确的是，标号函数 r 是用于证明执行顺序不存在循环互锁的拓扑排序判据，而非用于衡量调度性能的评分函数。通过该判定仅证明了调度时序在依赖关系上的无环可行性；方案在实际推进中是否会出现片上缓存超限或评测异常，仍须通过后续的离散事件模拟进行完整验证。

3.4 总体任务执行时间下界与候选剪枝准则

为了在受限时间内高效筛选候选解，我们需要建立问题固有的理论时间下界。

固定输入图 G 、核心总数 K 以及问题编号 q 。若对于集合 $\mathcal{F}_q(G, K)$ 中的任意合法方案 P ，其 Makespan 均满足 $M_q(P) \geq L$ ，则称 L 为该问题的**完成时间下界 (lower bound on makespan)**，其单位为时钟周期 (cycle)。

我们通过考察所有合法方案都必须完成的固有计算工作量与依赖链长，推导一个全局基础下界：

1. **算子执行周期定义：** 对于待调度的计算算子 $v \in V$ ，其执行时长定义为 $d_v = \max(1, \text{cycles}_v)$ (cycle)。该定义仅适用于具备确定计算周期的计算算子集合 V ，不适用于耗时取决于动态带宽的数据搬运操作；
2. **异构计算负载下界：** Cube 密集型算子总计算量记为 $W_M = \sum_{v \in V_M} d_v$ ，Vector 算子总计算量记为 $W_V = \sum_{v \in V_V} d_v$ 。由于全系统拥有 K 个核心，每个核心各拥有一条并行的 PIPE_M 与一条并行的 PIPE_V，在完全理想的零通信与无停顿假设下，全系统处理完矩阵与向量运算的最短周期数分别为 $\lceil W_M/K \rceil$ 与 $\lceil W_V/K \rceil$ 。由于两类 Pipe 属于物理独立的计算单元，不能将 $W_M + W_V$ 简单合并除以 K ；
3. **关键路径依赖下界：** 在由计算算子集合 V 及其直接与间接数据依赖构成的算子依赖图中，定义从起点算子到终点算子沿有向路径累加各算子时长 d_v 的最大值为**关键路径长度** L_{cp} (cycle)。由于同一依赖路径上的算子无论如何切分均必须依次串行执行，任何方案的完成时间不可能低于关键路径长度。

综合上述约束，针对全局所有合法方案成立的理论基础下界为：

$$L_{\text{basic}} = \max \left\{ L_{cp}, \left\lceil \frac{W_M}{K} \right\rceil, \left\lceil \frac{W_V}{K} \right\rceil, \max_{v \in V} d_v \right\} \leq M_q(P). \quad (3.5)$$

除了适用于全部方案的全局下界外，针对某一具体候选方案 $P = (\pi, a, \sigma)$ ，若考虑其特定的子图分配、各核心实际承担的算子工作量峰值以及跨核心传输所必需的最低延迟，可以推导出针对该方案的特定下界 $L(P) \leq M_q(P)$ 。

基于特定下界 $L(P)$ ，我们建立候选方案的**快速剪枝准则**：假设在搜索过程中算法已获得一份经过评测确认的合法方案 P_0 ，其 **Makespan** 记为当前性能上界 $U = M_q(P_0)$ 。对于新生成的候选方案 P ，若其特定下界满足

$$L(P) \geq U, \quad (3.6)$$

则该候选方案的实际完成时间绝不可能优于已有解 P_0 。算法可以直接舍弃该候选方案，从而省去对无潜力方案进行耗时的完整离散事件模拟调用。

若 L 为全局下界， OPT 为当前用例在给定核心数下的未知数学最优值，而 U 为算法所得最佳方案的实际 **Makespan**，三者无量纲上均为 **cycle** 且满足 $L \leq \text{OPT} \leq U$ 。当 $L > 0$ 时，同量纲相除得到无量纲的最优性相对间隙上界：

$$\frac{U - \text{OPT}}{\text{OPT}} \leq \frac{U - L}{L} = \frac{U}{L} - 1. \quad (3.7)$$

比值 U/L 直观量化了当前方案距离理论极限的逼近程度。比值越接近 1，表明所得调度方案越充分地挖掘了硬件算力潜能。

3.5 求解时间的组成与计算复杂度

算法的求解效率通过物理墙钟时间进行度量，它与模拟时钟周期（**Makespan**）具有本质不同的物理内涵。

求解程序的端到端运行时间 T_{solve} （以秒为单位）由以下五个互不重叠的执行阶段构成：

$$T_{\text{solve}} = T_{\text{parse}} + T_{\text{construct}} + T_{\text{check}} + \sum_{j=1}^b T_j + T_{\text{IO}}, \quad (3.8)$$

其中：

- T_{parse} 为输入计算图 JSON 文件的解析与数据结构初始化时间；
- $T_{\text{construct}}$ 为针对特定问题场景生成有限候选方案的算法构造时间；
- T_{check} 为算子覆盖合法性与扩展依赖图 $D^+(P)$ 无环性判定时间；
- b 为算法在求解当前用例期间实际调用的在线离散事件评测次数， T_j 为第 j 次在线评测的运行耗时；
- T_{IO} 为将最终优选方案持久化写入标准输出 JSON 文件的磁盘交互时间。

在计算复杂度层面，设待调度算子数为 $|V|$ ，算子间直接与间接数据依赖边数为 $|E_V|$ ，生成的子图数量为 $|S|$ 。算法在构造扩展图后的拓扑排序与无环性判定过程可在 $O(|V| + |E_V| + |S|)$ 的时间复杂度内完成。算法将单用例的在线评测调用次数 b 控制在极小常数范围（问题一至多 10 次，问题二与问题三至多 3 次），避免了组合爆炸式的时间开销，确保求解时间稳定在题面建议的高效区间。

4 问题一：子图对应的任务分别执行时的调度

4.1 任务划分的并行收益与通信开销

问题一中，每个子图所示的操作构成一个 Task。拆分能够把其中一部分计算交给另一个核心，但新 Task 之间的数据必须经过 DDR；即使它们最终仍分到同一个核心，也不能保留跨 Task 数据。因此，算法需要判断新获得的并行计算是否值得新增的搬运和等待。

设 Task s 在核心 k 的开始、完成时刻分别为 A_s, F_s 。记 $\text{prev}(s)$ 为同一核心上紧邻它的前一个 Task， $\text{pred}(s)$ 为它所依赖的 Task 集合。题面至少要求

$$A_s \geq F_{\text{prev}(s)} + 100, \quad (4.1)$$

$$A_s \geq F_t + 1000, \quad t \in \text{pred}(s), a(t) \neq a(s). \quad (4.2)$$

不存在的前驱不参与约束，时刻单位为 **cycle**。这些不等式说明拆分会增加哪些开始条件，却不能单独算出 $F_s - A_s$ ：Task 执行期间还可能与其他核心争用 DDR 带宽。仅把操作周期相加，也不足以得到其最终持续时间。

4.2 根据数据依赖结构构造调度方案

算法首先识别计算图中彼此没有计算操作之间的数据依赖的部分。把有依赖的操作间连线暂时看作无向边，能够沿连线互相到达的操作构成一个连通分量（**connected component**）。不同连通分量可以分别安排，但若它们读取同一外部输入，搬运仍可能相互影响。

固定实现最多产生六种基础方案，分别考虑：整体分配这些独立部分；处理工作量集中的较大部分及末端汇合计算；拆分工作量过多的部分；安排读取共享输入的的计算；分开能够独立计算的分叉支路；处理矩阵计算之后经过一段向量计算、再进入矩阵计算的受限链状结构。最后一类只在中间结果不被其他分支读取、并满足容量检查时采用。

在基础方案上，还可按固定条件增加三类比较：调整上述受限链的分组长度；尝试两种保持局部依赖结构的计算顺序；把符合条件的一个独立分支改分到另一核心。六份基础方案、至多一份分组长度调整方案、至多两份顺序调整方案和至多一份分支重分配方案，使在线 E1 评价总数不超过十次。不满足构造条件或与已有方案完全相同的候选不进入评价。

E1 根据完整方案评分，用于当前输入的方案选择；主结果由对应的 E0 原件确认。

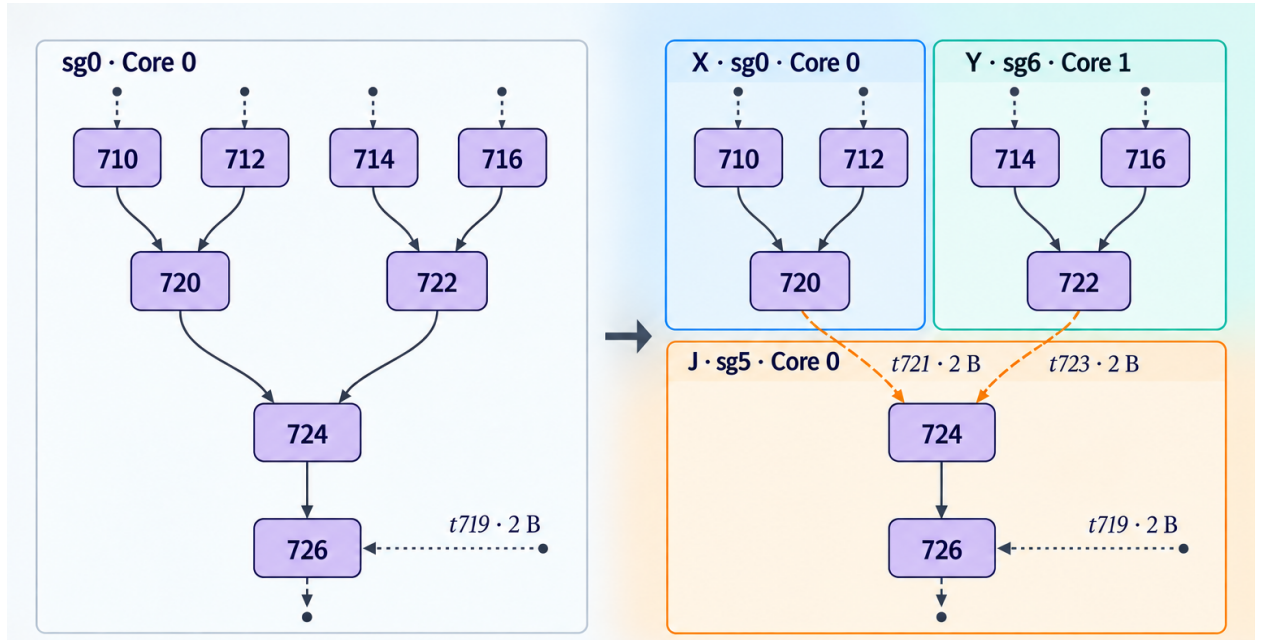


图 4.1 case_026、5 个核心的真实归约片段。左为原始划分，右为将独立支路改分到另一个核心；八个 ADD 节点的编号和依赖不变。点线端口连接裁剪范围外的输入或后继，橙色虚线为位于不同子图中的操作之间的依赖。

4.3 独立计算分支的重新分配及无环性证明

以下说明上一节最后一类改动。对原方案的子图依赖图，定义 $h(s)$ 为从起点到 Task s 的最长依赖路径边数；没有前驱时 $h(s) = 0$ 。算法只处理同一核心上的 Task 按 h 严格递增排列的方案，并要求同时参与改动、具有相同 h 的不同 Task 之间没有计算操作之间的数据依赖。

在工作量较大的一个 Task 内，找到两条互不依赖、随后汇合的计算支路。把相关操作划为三个互不重叠的集合： Y 为准备交给另一核心的独立支路， X 为留在原核心的其余前部计算， J 为等待两侧结果的汇合计算及其后续。要求 X 与 Y 之间没有依赖， J 不能返回依赖到 X 或 Y ，与外部 Task 相连的边仍遵守原来的 h 先后关系。

原来承担该 Task 的核心先后执行集合 X, J 中的操作；接收 Y 的核心先执行集合 Y 中的操作，再执行其原有的、具有相同 h 值的 Task。因此，程序必须确认接收核心原先存在这个 Task，而且在同一次修改中不会再从接收核心向其他核心转出支路。算法根据各核心的计算工作量，估计重新分配后最大的工作量是否严格减少。只有满足此条件，才继续生成需要完整评价的方案。

上述调整同时改变数据在哪个核心生成、哪些任务必须先完成。只看两条支路互不依赖，还不能排除它们与原有任务次序产生冲突。因而需要检查整个调整后的顺序，以下命题给出足够的条件。

命题 4.1。 在上述条件下，保留不同 h 之间的原顺序后，这种分支重新分配不会在加入同一核心顺序后的子图依赖图中引入有向环。

证明. 先按 h 的大小排列 Task。对 h 相同的 Task，将独立的 X, Y 放在前面，将接收核心原有的 Task 放在收到的 Y 之后，将 J 放在它所需的 X, Y 之后；若接收多个 Y ，再依实际次序编号。原方案中这些 h 相同的 Task 互无计算操作之间的数据依赖， X, Y 也互不依赖， J 没有返回边，且接收核心不同时向外转出支路。因此，在每个固定 h 内，数据依赖及同一个核心中的执行顺序都与这个编号一致。连接不同 h 的边则严格增加 h 。由命题 3.1，所得图无环。□

这个证明只说明执行顺序不会形成循环。新增子图仍可能增加复制，改变存储申请和操作等待。因此，形成完整方案后仍需检查全部操作覆盖与顺序，再用 E1 比较完成时间及代码规定的 COPY 统计量。

4.4 算法步骤与停止规则

算法 1：问题一的切分与调度。

输入：原图 G 、核心数量 K 、官方固定配置。

1. 按已固定的六类结构条件产生基础方案，删除内容完全相同的重复方案。
2. 比较需要评价的基础方案，保留当前最优的完整方案。
3. 若受限链状结构满足要求，增加一次分组长度调整并比较。
4. 按固定的两种规则分别调整局部计算顺序，检查并比较所得方案。
5. 若满足本章定义的高度与依赖条件，构造一次 X 、 Y 、 J 分组及分支重新分配。
6. 检查全部操作覆盖、每个子图只安排一次，以及完整顺序无环。
7. 仅按规定的完整评分接受改动，输出最佳合法方案；总 E1 调用不超过十次。

读取、覆盖检查和拓扑检查的时间与图中节点、边数量成正比；除此之外，还需执行工作量排序和可重分配支路查找。分支重新分配只对 Task 数不超过 320 的方案尝试，结构

检查设置 2 秒软预算：程序在规定检查位置发现超时后退出，并不保证在任意指令处都能立即中止。分支重新分配至多增加一次 E1 评价。调整分组长度的过程另有编译和子进程时间限制，均计入总求解时间。

4.5 实验结果与执行时间分析

固定版本在 100 张图、每图 1~5 个核心的 500 种组合上均得到合法结果。下表分别列出题目规定的平均加速比，以及使用算法输出方案计算的平均加速比；两行在一个核心时采用不同的方案。

核心数量	1	2	3	4	5
官方口径平均加速比	1.0000	1.9516	2.7514	3.4786	4.0553
优化方案 $\text{mean}(B/M)$	1.0021	1.9516	2.7514	3.4786	4.0553
改善/持平/退化	0/100/0	8/92/0	15/85/0	12/88/0	13/87/0
求解时间均值 / s	0.436	3.316	3.426	3.427	3.522

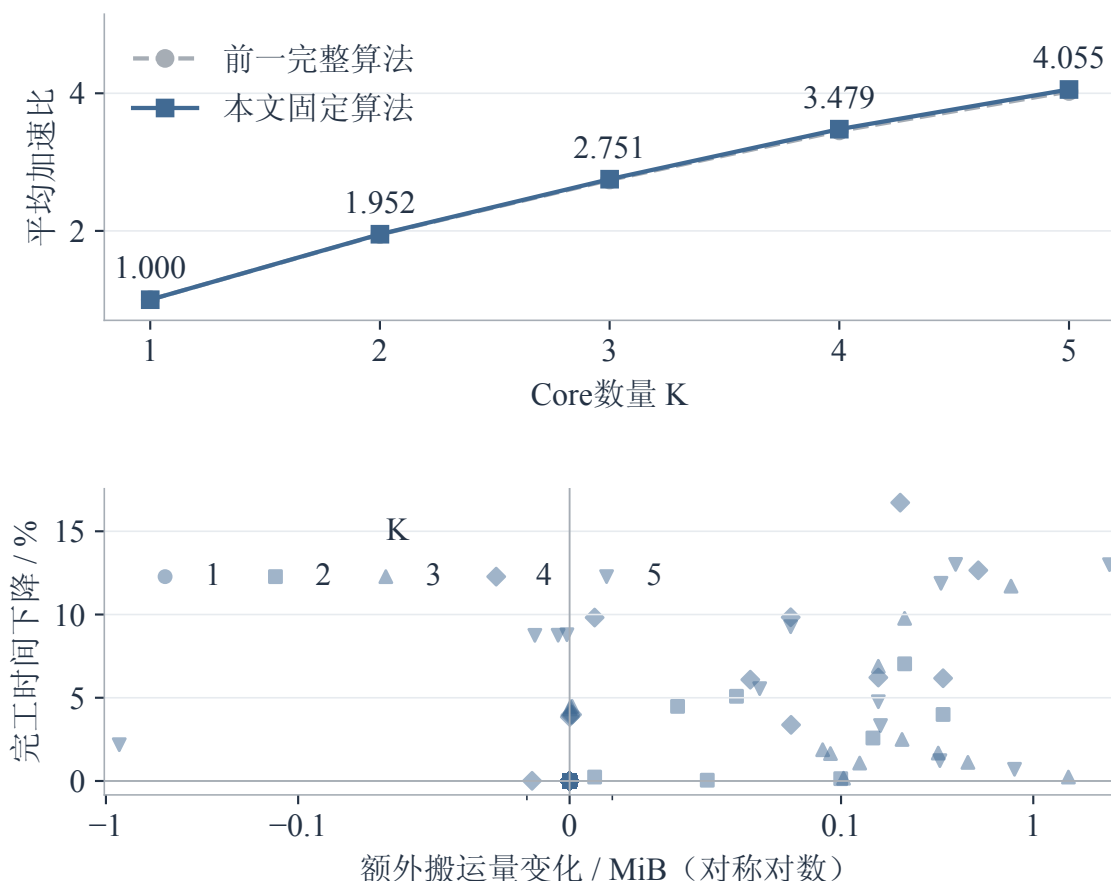


图 4.2 问题一的完整版本比较。上：每个核心数量的 100 图平均加速比；下：全部 500 种图与核心数量组合的时间与搬运量配对变化。核心数量用点形区分，横轴采用对称对数。

5 个核心时，平均加速比由前版 4.0259 提高到 4.0553，相对增幅为 0.729%。分支重新分配在全部 500 种组合中被选用 36 次，其余保留此前方案。上述版本比较同时包含其他规则的变化，尚未单独测得分支重新分配对全部改善的贡献。

5 个核心时的额外 DDR 搬运总量为 1053684536 B，比前版增加约 0.389%。因此这次版本改进缩短了完成时间，但没有同时减少总搬运。求解时间的 95% 分位数为 16.623 秒，最大为 25.552 秒。该批次在共享主机上用四个工作进程执行，没有清空操作系统缓存；这些时间不代表独占机器上的确定延迟。

全部测试包括 500 次新求解进程和 1130 次在线 E1 评价。48 种组合进行了新的 E0 评价；另外 452 种组合在方案文件、原图、核心数量、配置和评价规则逐项一致后，引用已有的 E0 结果。旧 E0 的运行时间没有记为新测值。

图 4.3 将修改前后的完整方案放在相同时间尺度上比较，可直接观察各核心上最后一个操作的完成时刻，以及不同 Pipe 上各操作的执行区间。

从这组配对结果可见，把独立计算交给另一个核心能够减少串行工作，但也要付出新的搬运和等待。问题二允许同一个核心保留不同子图间传递的数据，其边界搬运量需要按新的 Task 组成方式计算。

5 问题二：联合安排计算位置与数据搬运

5.1 同一核心内的数据复用

问题二把分配到同一个核心的各子图所对应的操作合并为一个执行任务 (Task)。先处理的子图中操作所生成的数据，可以留在这个核心的 L1/UB 中，供后面子图中的操作使用。如果生成和读取数据的操作分配到了不同核心，数据仍须通过 COPY_OUT 写入 DDR，再由目标核心的 COPY_IN 读入；目标读取须等待源 COPY_OUT 完成后 500 cycles。此时，把每个计算操作单独划成子图，也不意味着为每个操作建立独立 Task。提交文件中的子图顺序会影响官方程序在这个核心中安排操作的先后次序。

这种变化使分配与排序更加相关。把读取同一张量的操作安排到同一个核心，可能只需搬入一次；但如果两次使用相距较远，这块数据就要保存较长时间，期间可能妨碍其他数据分配空间。把这些操作分到不同核心虽然增加搬运，却可能让计算更早完成。本文分别估计可安排计算的时间和边界复制字节数，再比较完整方案。

5.2 利用执行资源空闲区间安排计算

算法记录每个核心上 Cube 和 Vector 已经安排的计算时间区间。安排下一个操作时，先根据保留的计算操作之间的数据依赖估计它最早能够开始的时刻，再寻找同类执行单元上足够长的空闲区间。这样，操作可以插到两次已安排计算之间，不必总是排到最后。

设 v 的估计最早开始时刻为 r_v ，计算时长为 d_v ； $\mathcal{I}_{k,p_v}$ 是核心 k 的 Pipe p_v 已经占用的时间区间的并集。选取最早的 s_v 使

$$s_v \geq r_v, \quad [s_v, s_v + d_v) \cap \mathcal{I}_{k,p_v} = \emptyset. \quad (5.1)$$

区间采用左闭右开表示，因此一次操作恰好在另一次完成时开始，不算同时占用资源。两条计算支路随后汇合时，算法联合比较它们的核心分配，至多检查 K^2 个核心组合，避免只看一侧较早完成而忽略另一侧。

这个估计没有完整模拟 DDR 带宽分配、缓存换入换出和存储空间分配次序。因此， s_v 用于候选构造，正式执行时刻由完整模拟确定；该估计不具备完成时间上界或下界的保证。算法最终用单操作子图及其列表顺序提交结果，由官方程序决定实际开始时刻。

同一个核心保留数据与在不同核心之间复制数据的区别见图 1.1。后续按这个区别计算边界复制量。

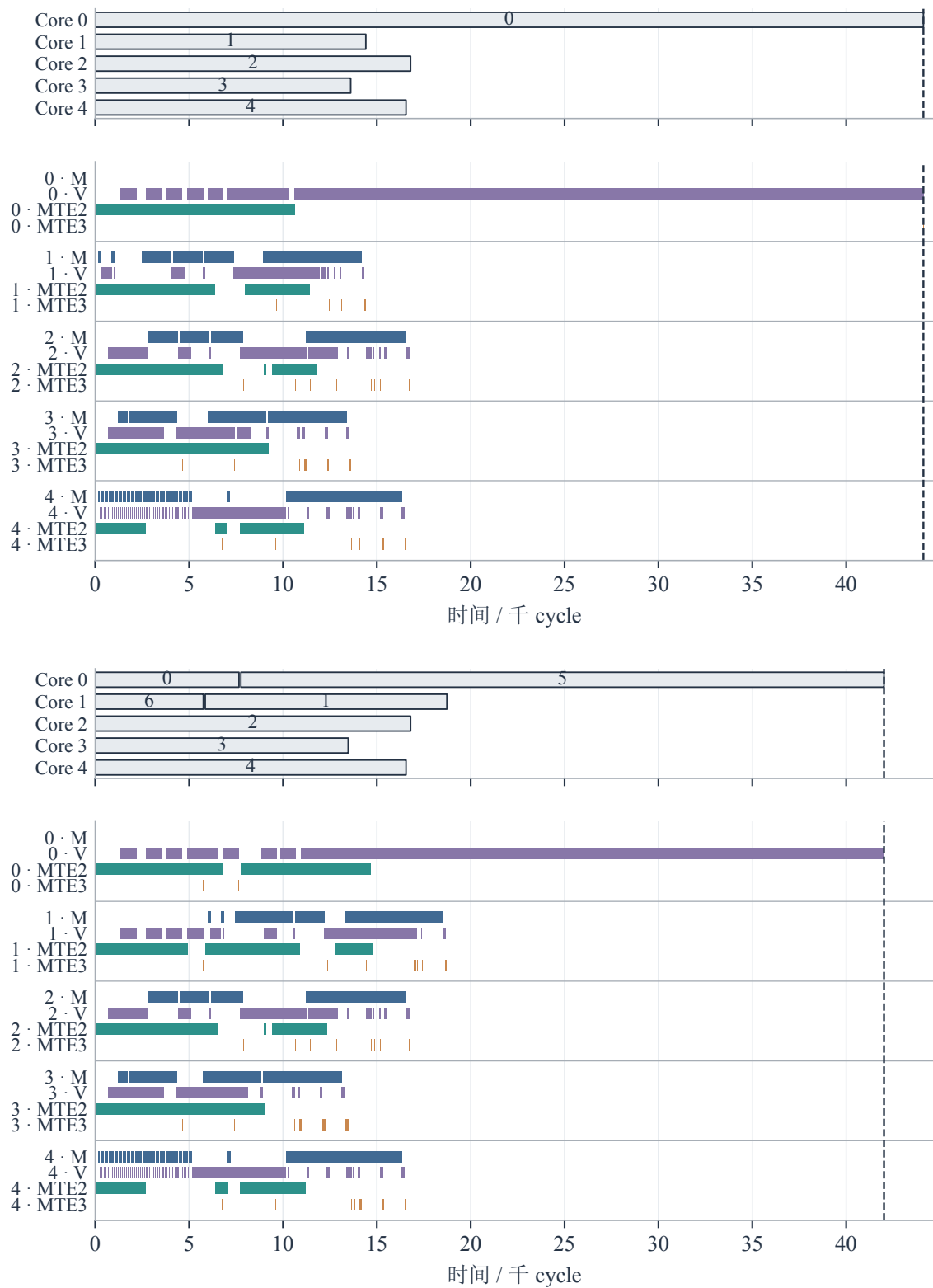


图 4.3 case_026、5 个核心两种切分的执行时间线。上：修改前方案， $M = 44,114$ cycles，额外搬运 211,680 B；下：分支重新分配后， $M = 42,014$ cycles，额外搬运 375,532 B。每组分别显示 Task 持续区间及四条 Pipe 的操作区间，横轴范围相同，数据来自官方模拟事件。

5.3 基于张量关联操作集合的复制量计算

先看一个简单例子。计算操作 u 生成 8 B 的张量 t ，操作 v_1, v_2 都读取 t 。如果两个读取操作分到同一个远端核心，就应按该核心需要的数据副本统计，而不能把两条读取关系视为互不相关的两次向另一个核心传递数据的需求。若只把 v_1 移回 u 所在的核心， v_2 仍需远端副本，相关边界复制量未必下降；将两个读取操作一起移回后，才消除这份远端需求。

为表示这种多操作共同使用一块数据的关系，引入**超边** (hyperedge)：它是一个顶点集合，可以同时包含两个以上顶点；由顶点与这些集合组成的结构称为**超图** (hypergraph) [3][5]。本文以计算操作为顶点，对中间张量 t 建立集合 $e_t = \{u, v_1, v_2, \dots\}$ ，其中包含生成 t 的计算操作和所有读取 t 的计算操作。超边只表示共同关联，不表示执行方向；原来的有向依赖仍单独保留。

应用以下公式前要求：全部非 COPY 计算操作已完整纳入模型，原图不同张量标识不通过逻辑标识合并，每个中间张量至多由一个原始计算操作生成，张量字节数为非负整数。这里的“生成”指计算得到数据，不把后来 COPY 得到的副本当成另一次原始计算生成。

给定核心分配 a ，定义 $\lambda_e(a)$ 为超边 e 中各操作所在的不同核心数量。令 w_e 表示新增一个相关核心所增加的边界复制字节数， C 表示与分配无关的固定复制量， $\mathcal{H}$ 为全部需计入的超边集合。在官方 Step2 插入容量不足所需的缓存换出、换入之前，复制量为

$$D_{\text{pre2}}(a) = C + \sum_{e \in \mathcal{H}} w_e (\lambda_e(a) - 1). \quad (5.2)$$

式中 D_{pre2}, C, w_e 的单位都是 B， λ_e 是无量纲整数。下标 pre2 表示这项复制量在官方 Step2 插入缓存换出、换入之前统计。

对 DDR 中的外部输入，只把读取该数据的计算操作放入相应集合。每增加一个需要该输入的核心，就增加 b_t B 的搬入量，所以 $w_e = b_t$ ，第一次输入复制计入 C 。对由计算产生的中间张量，集合还包含生成它的操作；官方边界构造对每个远端核心计入输出、输入一对复制，所以 $w_e = 2b_t$ 。最终输出等与分配无关的部分也计入 C 。两类数据对应的权重分别为 b_t 与 $2b_t$ 。

将上述公式用于 8 B 示例，当 u 在核心 0、两个读取操作在核心 1 时， $\lambda_e = 2$ ，该中间张量贡献 16 B 的边界复制。只移回一个读取操作，仍贡献 16 B；两个都移回后， $\lambda_e = 1$ ，该项变为 0。

还可以只更新受移动影响的超边。设一组操作 A 全部从核心 k 移至不同的核心 l 。对与 A 相交的超边 e ，令 $n_{e,k}, n_{e,l}$ 为移动前两个核心中属于 e 的操作数量。若目标核心原先没有相关操作，则增加一次权重；若源核心的相关操作全部被移走，则减少一次权重。因此

$$\Delta_e = w_e (\mathbf{1}\{n_{e,l} = 0\} - \mathbf{1}\{n_{e,k} = |e \cap A|\}). \quad (5.3)$$

$\mathbf{1}\{\cdot\}$ 在括号条件成立时为 1，否则为 0；没有与 A 相交的超边变化为 0。这个式子能够计算字节数变化，却仍未包括缓存容量不足引发的额外搬运，也没有描述计算与搬运能否同时进行。

5.4 计算工作量约束下的局部最小割方法

算法在上一节的时间区间构造结果上，选择两个核心及最多 16 条计算链组成局部调整范围。计算链指按依赖前后相接的一串计算操作；范围之外的操作以及必须保持位置的链不允许更换核心。其余操作在所选两个核心之间选择位置。

对于固定范围与固定操作，目标是减少这一范围影响的边界复制量。借助辅助节点，可以把两种核心选择表示为源点与汇点之间的最小割问题 (minimum source-target cut)：在

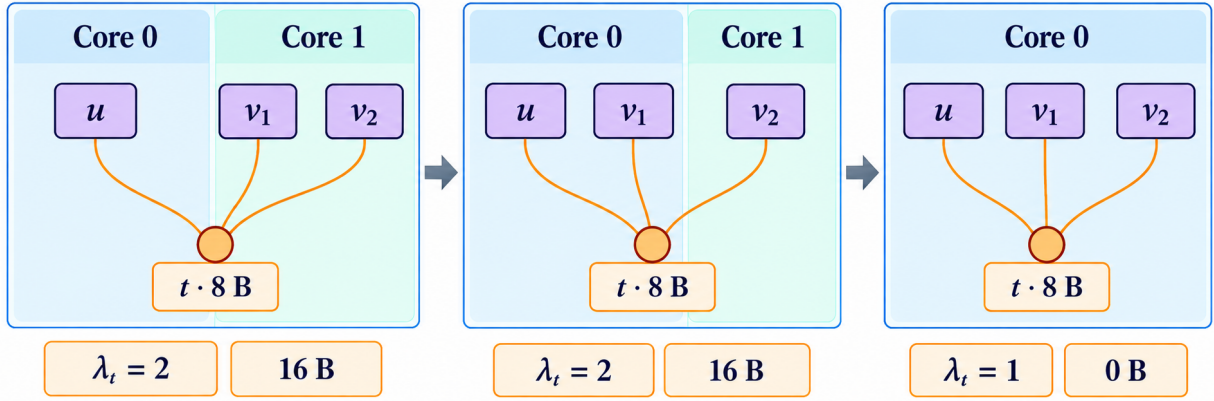


图 5.1 同一张量的计算操作分配。图中 u 生成 t , v_1, v_2 读取 t ; λ_t 表示这三个操作所在的不同核心数量。16 B、16 B、0 B 仅统计该张量的边界复制，线条只表示集合关联。

一个带非负边权的有向网络中，将指定的源点和汇点分到两侧，并使从源点一侧指向汇点一侧的边的权重和最小。辅助节点把“某条超边是否同时涉及两侧”转换为相应边权，因此求得的是当前局部复制字节的精确最小值。

只限制字节数可能把过多计算集中到同一核心。实现因而还规定每个核心、每类计算 Pipe 的累计计算时长上限。若新的分配超过上限，就固定部分移动链回到原核心，再求解缩小后的局部问题。原分配始终保留为可行选择；若局部有 r 条可调整链， $r \leq 16$ ，至多求解 $r + 1$ 次。接受前重新计算工作量与复制量，两项检查都通过才保留改动。

命题 5.1。 若每次固定操作都与原分配一致，每次最小割得到当前局部问题的精确最优值，且接受前核对计算时长上限，则被接受方案的局部 D_{pre2} 不大于原分配。

证明. 原分配在每次增加固定操作后仍可行，因此每次局部最优值都不大于原分配值。最终只接受满足计算时长上限且重新核算复制量不增加的方案，结论成立。□

命题保证的是固定局部范围内复制量不增加。固定部分操作会缩小可选分配集合，而 Makespan 还取决于操作次序和执行资源。算法保留调整前的完整方案，让最终评价决定是否采用新的分配。

5.5 完整算法及计算量

算法 2：问题二的切分与调度。

输入：原图 G 、核心数量 K 、官方固定配置。

1. 按图结构建立基础方案 P_0 。
2. 在各核心的计算空闲区间中安排操作，得到方案 P_1 ；条件不满足时保留 P_0 。
3. 对 P_1 中选定的两个核心和最多 16 条链，按边界复制量求局部最小割。
4. 检查各核心各 Pipe 的计算时长上限；必要时固定部分链回到原位置后重算。
5. 只有 Step2 之前的复制量严格减少，才根据新分配重新安排顺序，得到 P_2 。
6. 保留未修改的 P_1 ，按 P_0 、 P_1 、 P_2 的构造顺序去掉完全相同的方案。
7. 完整评价至多三份方案，先比较 Makespan，再按代码规定比较新增 COPY；同分保留较早方案。

这里 P_0, P_1, P_2 是本次求解中产生的方案编号，不是问题编号。当局部复制量没有减少时，不生成 P_2 ，保留 P_1 参与最终比较。

设图有 n 个计算操作、 m 条用于构造的计算操作之间的数据依赖，查找空闲区间累计扫描量为 Q 。若共调整 R 个局部范围，第 j 个范围的辅助网络有 n_j 个节点、 m_j 条边，含 $w_j \leq 16$ 条可调整链，一次最小割耗时为 $T_{\text{cut}}(n_j, m_j)$ ，则该部分总耗时不超过

$$\sum_{j=1}^R (w_j + 1) T_{\text{cut}}(n_j, m_j). \quad (5.4)$$

此外还需计入依赖识别、排序、空闲区间扫描 Q 及汇合附近至多 K^2 个分配比较。局部最小割不调用完整评价程序，完整在线评分最多三次。全部步骤的时间均计入求解时间。

5.6 实验结果与搬运量分析

固定版本在全部 500 种图与核心数量组合上得到合法结果。官方主曲线保留使用一个核心时的加速比为 1，同时单独列出算法改善仅使用一个核心的方案后的比值。

核心数量	1	2	3	4	5
官方口径平均加速比	1.0000	2.3167	3.2356	3.9711	4.5498
优化方案 $\text{mean}(B/M)$	1.2061	2.3167	3.2356	3.9711	4.5498
改善/持平/退化	59/41/0	64/36/0	54/46/0	49/51/0	42/58/0

与前一完整版本相比，268 种组合的 Makespan 缩短，232 种持平，没有变慢；69 张图至少在一个核心数量下得到改善。该比较评价的是完整算法在给定数据集上的表现；各局部方法的独立作用尚未由这组比较区分。

全部组合的额外 DDR 总量从 3043860416 B 增至 4159738560 B，增加约 36.66%。其中 55 种组合的时间与搬运同时减少，199 种时间减少而搬运增加，14 种时间减少、搬运不变，232 种两项均不变。算法选择以完成时间为主，因此会接受这类以更多搬运换取更早完成的方案。

平均求解时间为 4.029 秒，95% 分位数为 18.841 秒，最大 40.187 秒。独立 E0 评价的平均时间为 1.410 秒，95% 分位数为 6.592 秒，最大 13.980 秒。这批结果在共享 Mac 上用一个工作进程取得，整批约 46.63 分钟；每次求解新建进程，操作系统缓存保持原有状态。

5.7 减少搬运量后完成时间增加的反例

为检验只按容量和搬运量构造方案是否足够，已有实验对另一种固定构造 C04 评价了六个预先选择的 5 个核心用例。C04 是该实验的代码标识。六份方案都合法，且官方 Step2 未插入因容量不足产生的缓存换出、换入，但完成时间全部长于本章主算法。

case	主算法 / cycles	C04 / cycles	相对变慢
005	33515	38209	14.01%
016	2091375	2399125	14.72%
019	16247	28514	75.50%
025	942938	975658	3.47%
039	449653	494942	10.07%
075	343930	452120	31.46%

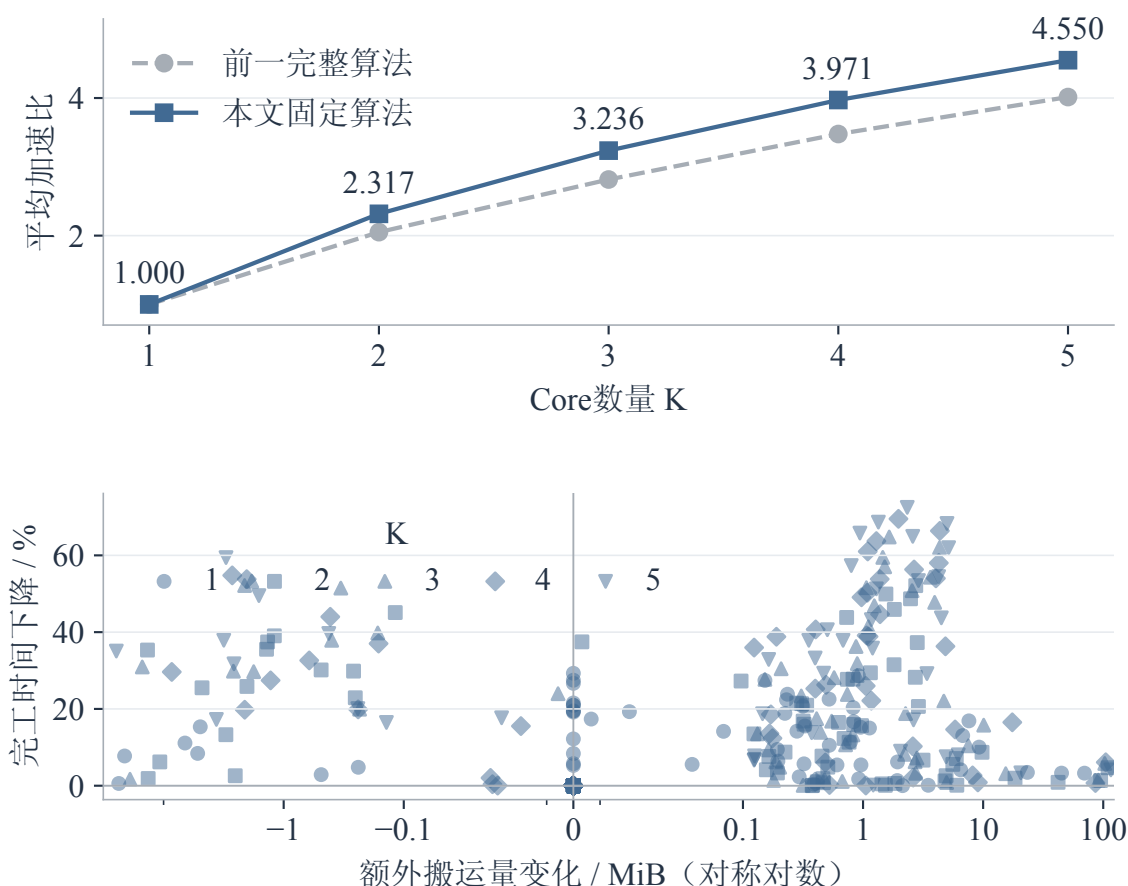


图 5.2 问题二的完整版本比较。上：按题目要求计算的平均加速比；下：500 种组合配对结果。部分时间改善伴随额外搬运增加，两项指标分别报告。

其中 case_019、case_039、case_075 的 COPY 字节减少，完成时间却增加。由这三个反例可知，减少搬运量且消除容量不足引起的换入换出，仍不足以保证完成时间缩短。具体原因还需结合执行事件分析。即使计算操作的顺序不变，搬运操作的顺序、不同核心之间传递数据所需的等待及 DDR 请求同时发生的情况仍可能改变。

我们在图 5.3 中对照两份方案的官方执行事件：搬运量较小的方案，最后一个操作反而完成得更晚。

本章用边界复制公式选择调整方向，再用完整模拟比较执行时间。问题三增加共享只读 Cache 后，还要判断读取请求到达时数据是否在 Cache 中，单看操作分到哪个核心更加不足。

6 问题三：安排计算顺序并利用共享只读 Cache

6.1 共享只读缓存的访问规则

问题三保留问题二的 Task 组成和分配到不同核心的操作之间的数据依赖，增加全部核心共享的只读 Cache。按官方代码，操作类型为 COPY_IN、能够确定搬入数据的逻辑标识且搬运字节数大于 0 时，操作开始就查询 Cache。程序使用这一逻辑标识查找数据。若数据存在，则为命中 (cache hit)，从 Cache 读取；否则为未命中 (cache miss)，从 DDR 读取，并在传输完成后把数据放入 Cache。FIFO 规则按插入时间选择需要移除的对象，命中不刷新该对象的次序。单份数据大于 Cache 容量时，不将它插入 Cache。

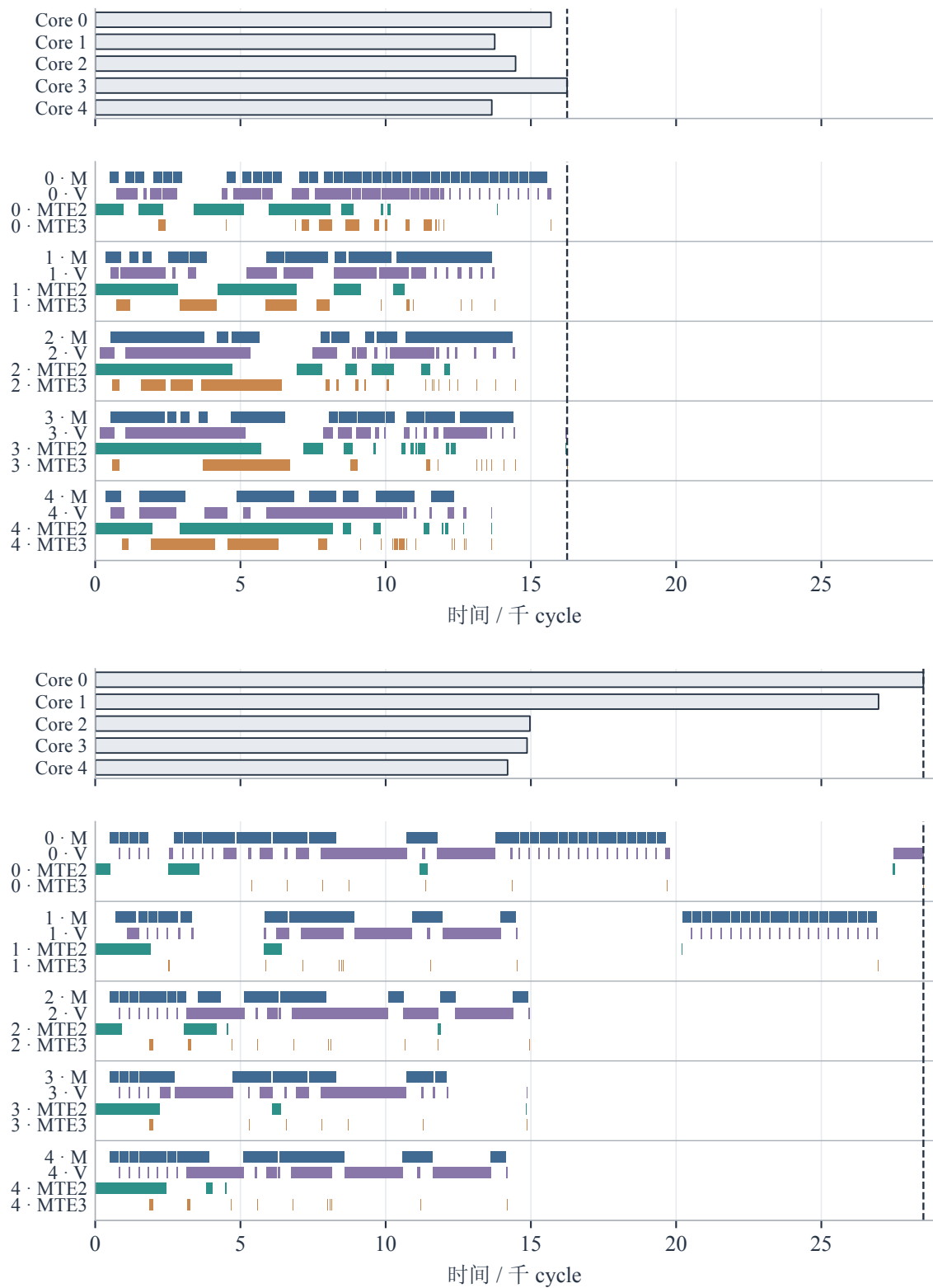


图 5.3 case_019、5 个核心的官方模拟的执行时间线。上：主算法， $M = 16,247$ cycles；下：C04， $M = 28,514$ cycles。每组分别显示 Task 和四条 Pipe，共用横轴范围。C04 的搬运量较少，且没有容量不足引起的缓存换入换出，但完成时间更长。

因此，前一次 DDR 传输尚未完成时，后续请求不能仅因数据正在传输就算作命中。即使命中，也仍须等待原有数据依赖及不同核心之间传递数据时的 500 cycles 等待条件满足，并占用 Cache 读取带宽。改变计算顺序会改变读取请求出现的时刻，而总命中字节数并不能完整描述这种时间关系。

本章算法在基础分配上，按输入结构尝试共享输入的访问顺序、符合条件的注意力计算顺序，以及下文定义的树状计算顺序。本章完整算法在代码中记为 Forest，其中树排序仅用于满足本节条件的输入。

例如，case_021、3 个核心的编译后张量 t1000000001 大小为 4096 B。官方事件给出以下生命周期；事件时刻为 cycles，表中间隔不表示等长处理阶段。

表 6.1 同一张量的 Cache 事件

时刻 / cycle	事件
0	COPY_IN 开始时未命中
207	DDR COPY_IN 完成，张量放入 Cache，记下插入顺序
4,956	另一个核心读命中，插入次序不变
323,850	插入张量 t1000000487 时，该最早插入的项被移除

6.2 子树处理顺序对结果存储峰值的影响

考虑两个互不依赖的计算分支 A、B。父操作必须等两者完成，才能读取它们的结果。A 先完成时，它留下的结果不能立刻释放，因为父操作还要等 B；这部分结果要与 B 计算期间的数据同时保留。反过来先计算 B，保留的数据量可能不同。

为了单独分析这个影响，先固定一棵树状计算结构：每个内部操作只输出一个张量，该结果只被唯一的父操作读取，其他树或分支不使用它。每次完整处理一棵子树后才处理下一棵，子树之间不交错执行。对子树 i 定义 h_i 为处理它时内部计算结果占用的最大总字节数， r_i 为子树完成后必须保留的输出字节数。这里统计的是此模型内的结果数据，不包括外部输入，也不分别区分 L1 和 UB。

例如，A 的 $(h_A, r_A) = (8, 2)$ KiB，B 的 $(h_B, r_B) = (7, 5)$ KiB，父输出为 3 KiB。先 A 后 B 时，三个需要检查的量为 8 、 $2 + 7 = 9$ 、 $2 + 5 + 3 = 10$ ，峰值为 10 KiB。先 B 后 A 时则为 7 、 $5 + 8 = 13$ 、 $5 + 2 + 3 = 10$ ，峰值为 13 KiB。最后一项保留了父输出先分配、子结果随后释放的规则。上述数值为用于解释公式的设定示例。

图 6.1 按这三个处理步骤分解所占空间，保留的数据与正在处理的子树分别着色。

6.3 非交错子树处理的最优顺序及证明

设同一父操作有若干子树， $\pi = (\pi_1, \dots, \pi_m)$ 为它们的处理顺序，父输出大小为 r_{parent} 。当处理第 j 棵子树时，先前子树的输出共占用 $\sum_{\ell < j} r_{\pi_\ell}$ ，当前子树最多再占用 h_{π_j} 。父操作开始分配输出时，各子树输出仍需保留。因此峰值为

$$H(\pi) = \max \left\{ \max_j \left(\sum_{\ell < j} r_{\pi_\ell} + h_{\pi_j} \right), \sum_i r_i + r_{\text{parent}} \right\}. \quad (6.1)$$

所有项的单位均为 B。该式允许比较排序，却不直接包含执行时间。

命题 6.1。 若 $h_i \geq r_i \geq 0$ ，在上述不交错处理子树的模型中，按 $h_i - r_i$ 从大到小排序能够使 H 最小。

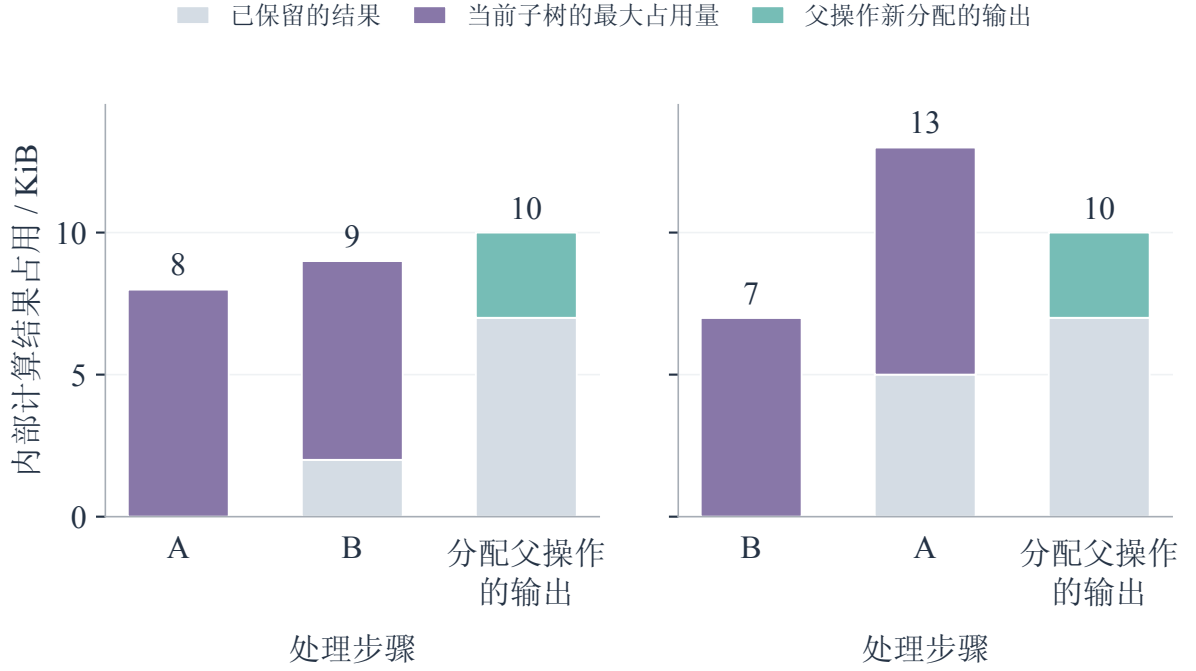


图 6.1 两种子树处理顺序的内部结果占用。左：先 A 后 B，峰值 10 KiB；右：先 B 后 A，峰值 13 KiB。横轴为处理步骤，不是模拟时钟；数字为第 6 章的公式示例。

证明. 取相邻两棵子树 A、B，假设 $h_A - r_A \geq h_B - r_B$ 。暂时不计两者之前已经保留的共同结果，A 后 B 与 B 后 A 的最大占用分别为

$$H_{AB} = \max(h_A, r_A + h_B), \quad H_{BA} = \max(h_B, r_B + h_A). \quad (6.2)$$

由假设可得 $r_A + h_B \leq r_B + h_A$ ，又因 $r_B \geq 0$ 而有 $h_A \leq r_B + h_A$ ，故 $H_{AB} \leq H_{BA}$ 。之前保留的数据只会给两种次序各加同一个常数；两棵子树都完成后，各自留下的结果之和同为 $r_A + r_B$ ，后续计算及父输出分配不受交换影响。不断把次序不符合要求的相邻子树交换，最大占用不会增加，最终得到按 $h_i - r_i$ 递减的顺序。对子树自下而上应用这一结论即可。□

这种通过交换相邻分支证明存储次序的方法，与表达式树代码生成中的经典研究相关 [4]。针对本题按字节给定大小的结果数据，我们以上述相邻交换论证证明相应结论。

算法先确认每个内部输出只被父操作读取、原 COPY 只位于允许的边界，并且依赖中的数据大小完整，再自下而上计算 h_i, r_i 。按所求顺序先处理子节点、后处理父节点，保持原来的核心分配，用单操作子图列表提交新顺序。

该最优性结论适用于前述单棵树、内部结果、子树不交错处理的模型。完整模拟还计入外部输入、不同树的最终结果、L1 与 UB 的独立容量以及新建数据副本；算法据此进一步评价新顺序的缓存换入换出和完成时间。

6.4 固定候选方案的完成时间下界

当一份候选的单操作子图、核心分配和各核心各 Pipe 顺序已经固定时，可以列出每次成功执行都必须满足的时间不等式。对必要的先后关系 $u \rightarrow v$ ，写为

$$s_v \geq s_u + d_u + \delta_{uv}. \quad (6.3)$$

s_u, s_v 是开始时刻, d_u 是计算时长, δ_{uv} 只计入能够证明不可避免的等待, 单位均为 cycle。同一对操作之间若有多个条件, 应取最大的必要等待, 不把重复条件相加。

将这些关系组成有向图, 其最长路径长度 $L(P)$ 不大于固定候选的实际完成时间。如果当前已成功评价方案的完成时间为 U , 且 $L(P) \geq U$, 该候选就不能严格改善, 可以省去一次 E0 评价。

上述下界由每次合法执行均满足的时间不等式推得。放宽约束后, 任意一个贪心调度结果仅是一份可行安排的完成时间; 只有求得该放宽问题的最优值, 或另证必要不等式, 才能据此取得下界。

6.5 算法步骤与复杂度

算法 3: 问题三的切分与调度。

输入: 原图 G 、核心数量 K 、官方固定配置。

1. 按结构与计算空闲区间构造基础方案, 并进行评价。
2. 对满足条件的输入, 尝试连续安排共享输入访问或调整注意力计算的次序。
3. 在剩余评价次数允许时, 识别本章定义的树状计算, 按子树峰值减去输出字节数的差值排序, 产生方案。
4. 跳过与已有方案完全相同的结果; 对满足证明条件的候选计算完成时间下界。
5. 若下界不小于当前最佳完成时间, 则跳过完整评价。
6. 否则调用 E0, 只有 Makespan 严格缩短才替换当前方案。
7. 一次求解共用至多三次 E0 预算, 输出最佳成功方案及调用记录。

全部候选共用三次 E0 调用预算。若前面的候选已使用全部次数, 后面的树排序就不会追加评价。结构不符合、方案重复、下界无法改善、合法性不通过和评价后未改善分别作为候选停止原因记录。

给定含 n 个节点、 m 条依赖的树状结构, 识别及按子先父后遍历需要 $O(n + m)$ 时间。每个父节点对孩子排序, 合计需要 $O(\sum_v \deg(v) \log \deg(v))$ 时间, 其中 $\deg(v)$ 为其孩子数。候选约束图含 m' 条必要边时, 最长路径可在 $O(n + m')$ 时间内求得。完整算法的计算量还包括基础分配和至多三次官方评价。

6.6 启用和关闭共享缓存的配对实验

Forest 固定版本完成全部 500 种图与核心数量组合, 均得到官方成功结果, 500 次求解共调用 973 次在线 E0, 没有重试。与前一完整策略相比, 24 种组合的 Makespan 缩短、476 种持平, 没有变慢。额外 COPY 和因容量不足产生的搬运分别有 18 种减少、479 种持平、3 种增加, 两种统计各净减少 413450240 B。其中, 因容量不足产生的搬运属于额外 COPY 的一部分。

为比较 Cache 作用, 对每份最终方案分别运行问题二、三的官方评价。下表“无 L2”是同一份 P3 方案关闭共享 Cache 后的结果, 并不是问题二主算法生成的另一份方案。

5 个核心的逐图 Cache 加速比平均为 1.0829。这个数值按每张图的 M_2/M_3 计算后取平均。按所有访问字节汇总后的命中率为 47.45%, 逐图命中率算术平均则为 31.36%; 前一种统计让访问字节较多的图占更大权重。

平均求解时间为 4.401 秒, 中位数 1.616 秒, 95% 分位数 20.009 秒, 最大 51.335 秒。这些时间包含求解过程中调用 E0 的时间, 来自共享主机四个工作进程的原始批次;

核心数量	无 L2: $\text{mean}(B/M_2)$	Cache: $\text{mean}(B/M_3)$	$\text{mean}(M_2/M_3)$	逐图字节命中率均值
1	1.1971	1.1998	1.0023	6.47%
2	2.2949	2.3120	1.0079	16.25%
3	3.1745	3.2587	1.0314	25.96%
4	3.9211	4.0915	1.0512	24.53%
5	4.4763	4.7576	1.0829	31.36%

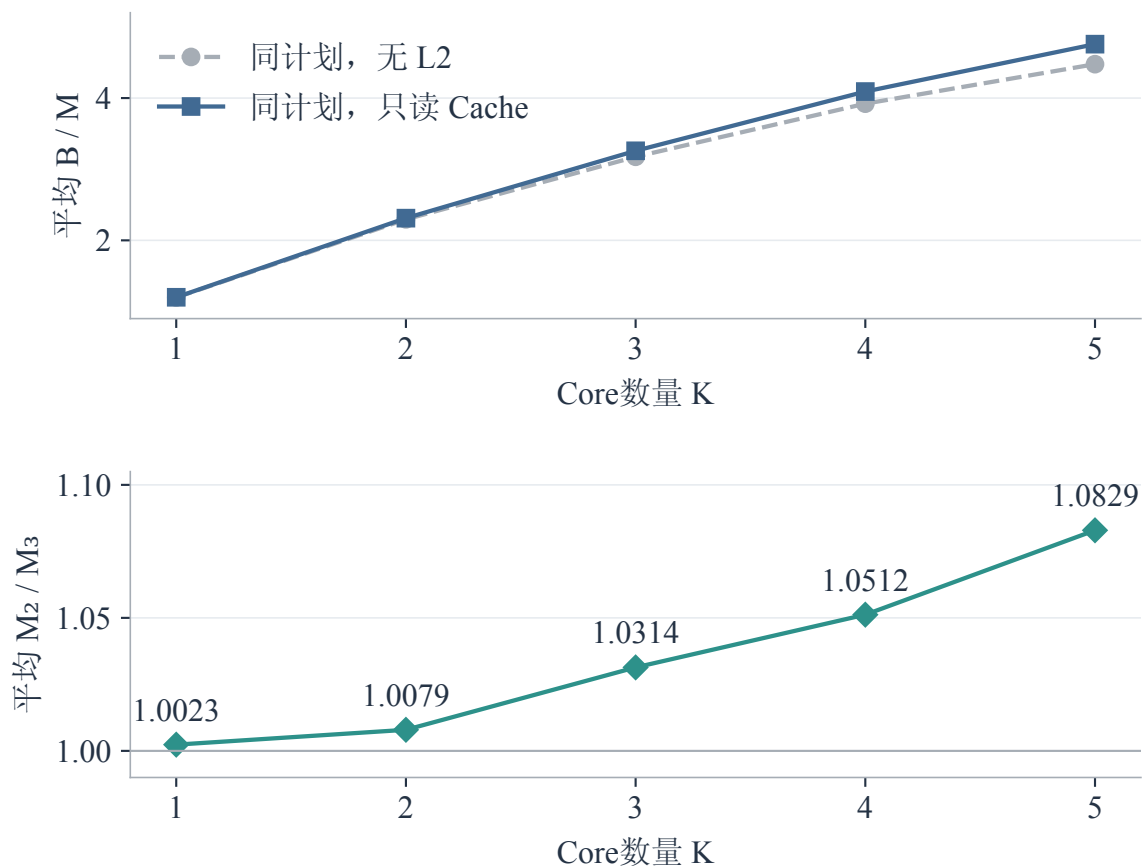


图 6.2 问题三的保持方案不变的配对评价。上：无 L2 与只读 Cache 的平均基线加速比；下：逐图 M_2/M_3 的均值，每种核心数量包含 100 对结果。

6.7 附加比值条件对方案选择的影响

已有后续实验 R9F 对 5 个核心候选增加了三个同时满足的接受条件：启用 Cache 后的 M_3 严格下降，关闭 Cache 后的 M_2 不增加，且比值 $G = M_2/M_3$ 不下降。R9F 是该实验的代码标识，这三个条件不是官方评分要求。

100 张 5 个核心图中，96 张不符合该构造条件，1 张由候选下界排除，另外 3 张被上述联合条件拒绝，没有方案被接受。最终 500 份输出文件均与 Forest 一致。然而，被拒绝的三份候选实际上同时缩短了 M_2 和 M_3 ：

因为完成时间为正，保持 G 不下降等价于

$$M_{2,\text{new}}M_{3,\text{old}} \geq M_{2,\text{old}}M_{3,\text{new}}. \quad (6.4)$$

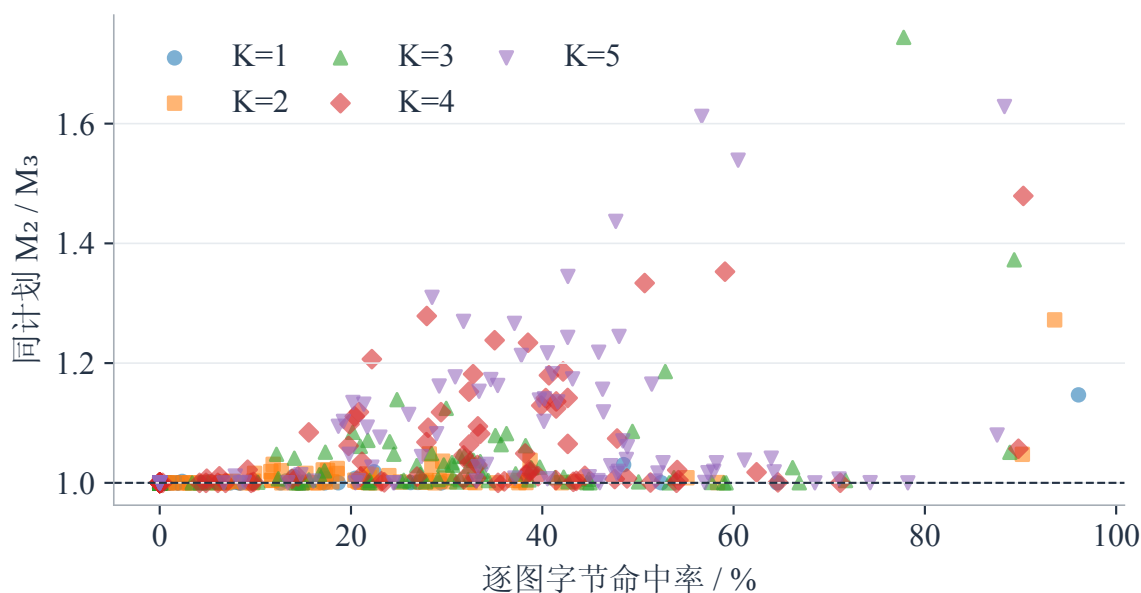


图 6.3 500 个保持方案不变的配对结果中的字节命中率与 Cache 收益。保留低于 1 的负收益点；100 个用例的不同核心数量结果相关，不能当作 500 个独立随机样本。

case/K	原 M_2	候选 M_2	原 M_3	候选 M_3
064/5	7029	6899	6981	6877
068/5	131631	98913	116345	97971
088/5	97305	66729	85789	66224

当无 Cache 的时间改善得更快时，这个比值可能下降，即使启用 Cache 后的时间也明显缩短。在这三份候选中，比值条件拒绝了 Makespan 更短的方案，因而妨碍了主要目标的改善。

本节将 R9F 作为附加选择条件的对照实验，主表报告 Forest 的完整结果。R9F 分两批执行 400 和 100 种组合，共使用 982 次在线 E0，仅用于说明这项额外选择条件的实际影响。

在限定的树状计算模型中，所证排序使内部结果的存储峰值最小；在完整执行中，算法则根据官方模拟的完成时间选择方案。

7 实验结果与分析

7.1 输入、对照和统计方法

实验采用题面给出的全部 100 张原始计算图，核心数量分别取 1~5，每题共 500 种输入组合。三题各使用一个固定的完整求解器版本，方案构造条件和评价次数上限在运行前确定。附录 C 列出各题固定程序版本；`data/source-receipt.json` 记录输入、方案和评价结果的来源，`data/all-results.csv` 汇总逐例数值。

全文使用三类比较。第一类用官方仅使用一个核心时的基线除以当前方案的完成时间，得到题目要求的加速比。第二类比较相同题目、相同图和相同核心数量下两个完整算法版本，观察方法修改后的总体变化。第三类保持问题三的方案不变，只切换共享只读 Cache，

比较能否从共享只读缓存读取数据对同一方案完成时间的影响。三类比较分别评价相对单核心基线的收益、算法版本变化的收益及共享缓存的收益。

所有结果表格和数据图均由同一份 1500 行结果表生成。导出时检查每题 100 图与五种核心数量是否完整、基线是否对应原图、问题三的两份结果是否使用同一方案，再重新计算平均值。

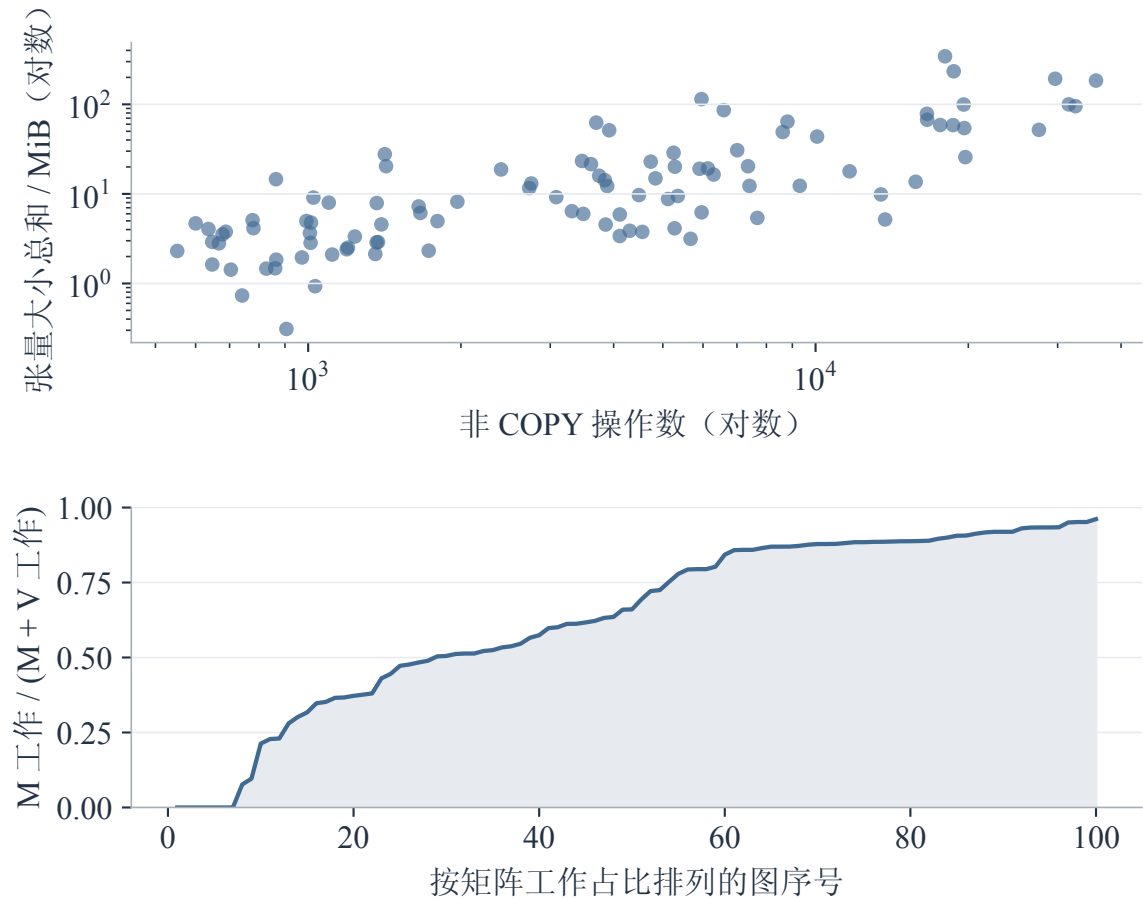


图 7.1 官方 100 张计算图的结构统计。上：非 COPY 操作数与张量大小总和；下：按矩阵计算的时长占两类计算总时长的比例排序的用例分布。

7.2 三问的主要结果

下表报告平均加速比。先对每张图计算“官方单核心基线的完成时间 / 算法方案的完成时间”，再对 100 张图取算术平均；三问使用同一组官方基线。为了展示算法还能怎样改变仅使用一个核心的方案，表内保留实际使用一个核心时的比值；问题一、二要求的主曲线仍按题面在使用一个核心时取 1。

核心数量	P1 优化方案	P2 优化方案	P3 优化方案	P3 同计划无 L2
1	1.0021	1.2061	1.1998	1.1971
2	1.9516	2.3167	2.3120	2.2949
3	2.7514	3.2356	3.2587	3.1745
4	3.4786	3.9711	4.0915	3.9211
5	4.0553	4.5498	4.7576	4.4763

问题一要求不同 Task 之间的数据经过 DDR；问题二、三允许同一个核心保留在不同子图之间传递的数据，但仍受带宽、容量和不同核心之间传递数据所需的等待限制。三问之间的比较同时涉及硬件规定和算法的变化。

“P3 同计划无 L2”一列使用问题三算法生成的方案；“P2 优化方案”由问题二算法单独求得，两者不是同一个调度结果。5 个核心时，P3 两列平均基线加速比之比约为 1.0628，而逐图 Cache 加速比的均值是 1.0829。差异来自先求比值还是先求平均的顺序，本文以第 2 章定义的逐图比较为准。

7.3 求解时间与评价调用次数

求解时间包含输入读取、构造与检查、在线方案评价及结果写出。问题一、二在求解过程中使用 E1、E2，最终 E0 复评另计；问题三在求解过程中直接使用 E0，其时间已经包含在求解总时间内。

方法与样本范围	在线评价总次数	求解均值 / s	P95 / s	最大 / s	运行条件
P1, 100 图、5 核心	全部 500 种组合 1130 次 E1	3.522	16.623	25.552	共享主机, 4 个工作进程
P2, 全部 500 种组合	1131 次 E2	4.029	18.841	40.187	共享主机, 1 个工作进程
P3 Forest, 全部 500 种组合	973 次 E0	4.401	20.009	51.335	共享主机, 4 个工作进程

表中的工作进程数说明同一批次同时运行多少个求解进程。主机还可能还有其他任务，因此不同批次的时间不能直接用作独占硬件的严格速度排名。问题一的时间一栏只统计 100 张图的 5 个核心结果，调用次数则覆盖全部 500 种组合，两者范围不同。

95% 分位数 (95th percentile, P95) 表示按时间排序后接近 95% 位置的值。问题一原始报告采用线性插值，问题二、三采用向上取整得到的第 $\lceil 0.95n \rceil$ 个次序值； n 是样本数量。各方法的 P95 按相应原始报告的定义计算。

本批次所有成功求解进程的实测时间均低于题面建议的 5~10 分钟。测量范围限于上述共享主机；独占硬件、清空操作系统缓存及跨平台运行尚未测试。

7.4 算法改进的实验依据与适用范围

完整算法版本之间的比较，能说明修改后整体结果如何变化；某个构造被选中的记录，能说明该构造确实影响了输出；同一用例的结构图与时间线，能说明该次运行发生了什么。若多个部分同时修改，仅有前后版本比较还不能分别确定每一部分贡献多少。

问题一 5 个核心时平均加速比提高约 0.729%，额外 DDR 总量增加约 0.389%。问题二有 268 种组合缩短完成时间，但总额外 DDR 增加约 36.66%。在这些输入中，完成时间缩短与搬运量增加同时出现。计算与搬运重叠所产生的具体贡献仍需逐项分析执行事件。

问题三完整版本有 24 种组合的完成时间缩短，同时有 3 种组合的额外 COPY 增加。C04 在所测试的六个用例上均变慢；这个结论限于这六份方案。R9F 中的三份候选虽然缩短了完成时间，却因附加的比值条件被拒绝。

7.5 完成时间的下界与最优值的偏差

如果一个时间下界 L 适用于该输入的全部可行方案，而当前方案完成时间为 U ，则 $U/L - 1$ 给出相对最优差距的保守上限。问题二已有固定版本的下界核对记录：在 1~5 个核心下，分别有 81、62、32、33、23 张图满足 $U \leq 1.05L$ 。在记录所声明的条件下，这些结果距最优不超过 5%。这是逐输入的数学保证，不是统计显著性检验。

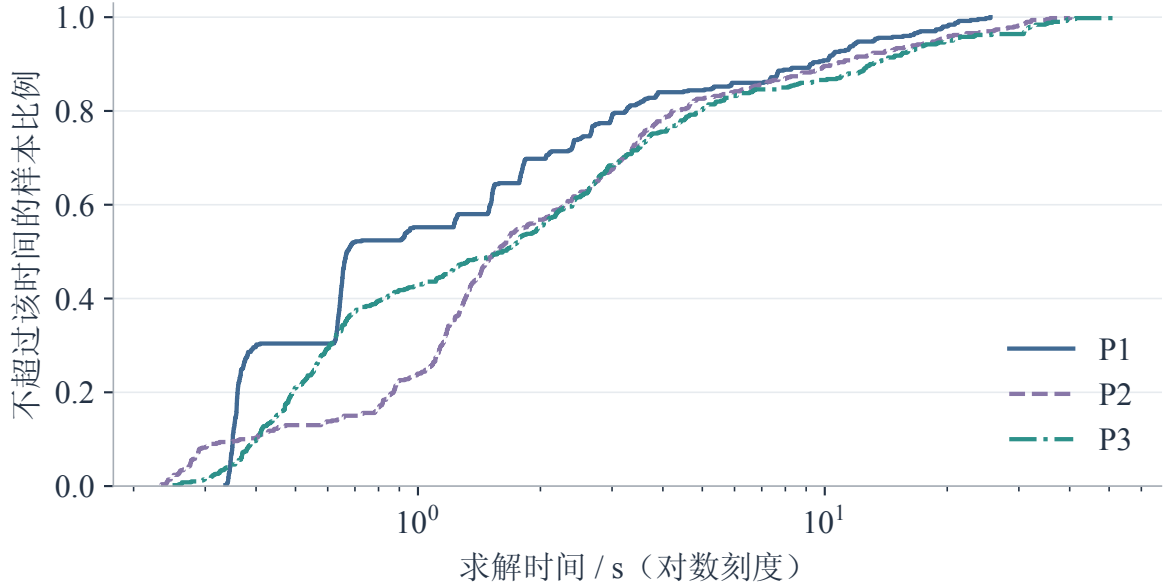


图 7.2 三种求解程序的实际经过时间累计分布，每题 500 次。时间包含方案构造及在线评价，横轴采用对数刻度；批次资源不同，不据此比较独占机器的速度。

5 个核心时，平均 B/L 约为 6.108，高于算法达到的 4.550。下界忽略部分通信和容量约束时，差距也会增大，因此 6.108 不是已经证实可以达到的平均加速比。附录进一步分析工作量与必要等待怎样共同形成下界，但没有为这些新推导重新运行全量核对；本节仍使用已有固定结果。

7.6 实验范围与局限

全部 100 张官方图都参与过研发，结果只说明给定公开数据集上的表现，不构成对未见输入的验证。同一图的五种核心数量结果相互关联，不能直接当作 500 个独立随机样本估计显著性。按结构而非用例编号选择算法分支，能够避免显式按编号查表，但不能消除研发过程中接触公开图的影响。

三个主版本均在给定输入上完成规定的全部组合，尚未测试更广范围的输入。方案质量由官方模拟的时钟周期衡量，求解效率由本机程序运行秒数衡量；本次未开展真实 NPU 设备实验。

8 方法评价与结论

8.1 主要方法与适用范围

本文共同处理计算图切分、核心分配和子图顺序三个决策。问题一把同一核心负责的两组独立计算分配到不同核心，使两组操作能够同时进行，并检查新增 Task 造成的等待与搬运。问题二根据生成、读取同一张量的操作分布计算边界复制量，并在计算执行单元的空闲区间中安排操作。问题三对符合条件的树状计算选择较少占用内部结果存储空间顺序，再检查共享只读 Cache 条件下的完成时间。

三问的数学分析分别回答明确的问题。沿全部子图数据依赖边和同一核心上的任务顺序边严格递增的编号，证明这些先后关系不形成有向环；张量关联集合说明为什么同时移动多个读取操作可能减少复制，而逐个移动没有立即收益；相邻子树交换证明在限定模型

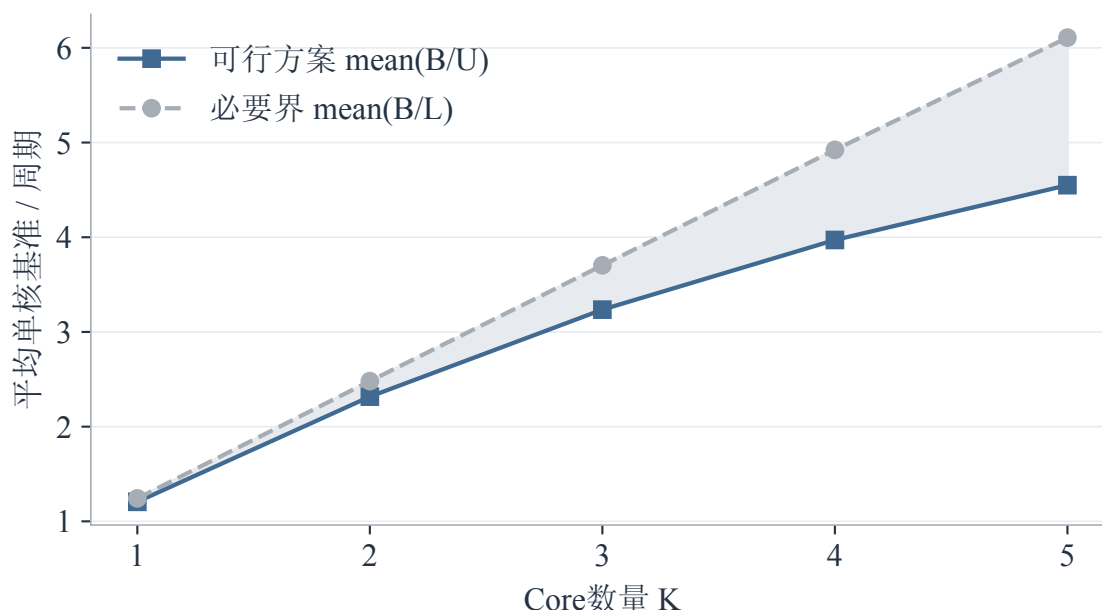


图 7.3 同一输入、配置与方案范围下的时间上界和下界。 $L \leq \text{OPT} \leq U$ ，阴影是必要界给出的上限与当前结果之间的差距，不代表一定可实现的收益。

中怎样排列能够减小存储峰值；完成时间的必要下界则排除不可能改善的固定候选。这些结论都有具体条件，不代替完整执行评价。

算法还有三方面限制。第一，共享带宽、缓存插入移除和存储空间分配会相互影响，目前的局部表示不足以完整预测全部执行行为。第二，树状计算排序等方法依赖特定输入结构；若一个结果被多个分支读取，或输入存在不满足公式要求的标识关系，就不能直接套用原证明。第三，已有实验仍发现局部更快但被附加条件拒绝的方案，表明接受规则还可以改进。后续版本的整体收益有待重新进行全量评价。

8.2 结论

三个固定算法均完成 100 张图、每图 1~5 个核心的官方评价。5 个核心时，问题一、二、三的平均基线加速比分别为 4.0553、4.5498、4.7576；保持问题三方案不变，启用 Cache 相对关闭 Cache 的逐图平均加速比为 1.0829。结果来自统一输入与冻结配置，搬运增加和候选退化也一并报告。

这些结果支持本文的求解方法：先从原图依赖和数据使用关系构造少量方案，用证明检查必要条件，再以完整模拟决定性能是否改善。三问新增的硬件能力不同，但都需要明确区分数据量、执行时间和程序求解时间。后续工作将针对已出现的反例调整分配与接受规则，并在新的固定版本上验证。

参考文献

- [1] 第二十三届中国研究生数学建模竞赛. A 题：通用神经网络处理器下的多核调度问题及官方附件 [Z]. 2026.
- [2] GRAHAM R L. Bounds for Certain Multiprocessing Anomalies[J]. Bell System Technical Journal, 1966, 45(9): 1563–1581. DOI: 10.1002/j.1538-7305.1966.tb01709.x.

- [3] ÇATALYÜREK Ü V, AYKANAT C. Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication[J]. IEEE Transactions on Parallel and Distributed Systems, 1999, 10(7): 673–693. DOI: 10.1109/71.780863.
- [4] SETHI R, ULLMAN J D. The Generation of Optimal Code for Arithmetic Expressions[J]. Journal of the ACM, 1970, 17(4): 715–728. DOI: 10.1145/321607.321620.
- [5] BLACK P E. Hyperedge; Hypergraph[EB/OL]. Dictionary of Algorithms and Data Structures, NIST. [2026-09-26]. <https://xlinux.nist.gov/dads/HTML/hyperedge.html>.

附录 B 补充数学推导

B.1 数据依赖所给出的完成时间下界

本节假设每个需要讨论的原图中间张量至多由一个原始计算操作生成，不同原始张量标识不合并成同一逻辑数据标识。取出原始计算操作，只保留有依据的先后关系，包括原来就存在的计算关系，以及同一张量的生成操作必须先于读取操作的关系。不任意缩短经过原始 COPY 的路径。

在任何官方成功执行中，这些计算操作及其时长都被保留，每个核心的每条 Pipe 也仍同时至多处理一个操作。直接取该次执行中各计算的核心和开始、完成时刻，就得到满足保留关系的一组安排。因此，在这些必要关系下求得的最小必要时间，不会大于官方完成时间。

对于分配到不同核心的计算操作，必须核对两者之间的依赖怎样在官方执行中保留。问题一要求相关前驱 Task 完成后，后续 Task 才允许开始；问题二、三要求源 COPY_OUT 完成，再经过 500 cycles，目标 COPY_IN 才可参与就绪判断；实际开始仍需满足其余数据依赖、执行单元和存储条件。接收端若因容量不足而需要再次读入数据，还受官方 Step2 次序和 MTE2 操作队列顺序限制，不能跳过所需的基础 COPY_IN。上述必要关系均从官方成功执行所满足的条件中提取。

B.2 指定时间区间内的工作量下界

在 B.1 的计算操作之间的数据依赖图上，为操作 v 定义两个不包括自身的时长： h_v 为它之前最长依赖路径的时长， t_v 为它之后最长依赖路径的时长。无相应前驱或后继时取 0。递推式为

$$h_v = \max_{u \rightarrow v} (h_u + d_u), \quad t_v = \max_{v \rightarrow w} (d_w + t_w). \quad (\text{B.1})$$

本节 h_v 以 cycle 计，与第 6 章按字节定义的子树存储量 h_i 是不同的局部记号。取 $a, b \geq 0$ ，选定一种 Pipe p ，考虑非空操作集合

$$S = \{v : p_v = p, h_v \geq a, t_v \geq b\}. \quad (\text{B.2})$$

任何成功执行中， S 的操作都只能完整安排在 $[a, M - b]$ 内：它们之前至少需要 a 个周期，之后至少需要 b 个周期。同类 Pipe 共有 K 个，每个操作又不能拆成多段交给不同核心。因此，除了总工作量，还可以利用操作不可分割这一条件。

将 S 中 n 个操作的时长降序排列为 $d_{(1)} \geq \dots \geq d_{(n)}$ 。定义

$$A_K(S) = \max_{\substack{r \geq 1 \\ m=(r-1)K+1 \leq n}} \sum_{j=m-r+1}^m d_{(j)}. \quad (\text{B.3})$$

于是

$$M \geq a + b + \max \left\{ \left\lceil \frac{\sum_{v \in S} d_v}{K} \right\rceil, A_K(S) \right\}. \quad (\text{B.4})$$

证明. 令 $x = M - a - b$ ， K 个同类 Pipe 在该区间内最多提供 Kx 个计算周期，得到第一项。取前 $m = (r-1)K + 1$ 个操作，至少一个核心要执行其中 r 个；即使选其中最短的 r 个，也至少需要 $\sum_{j=m-r+1}^m d_{(j)}$ 周期。对允许的 r 取最大得到第二项。两项都限制同一个区间的长度，故取最大值，不相加。最后只加一次前后所需的 $a + b$ 。□

例如，两个核心执行三个不可拆分操作，时长分别为 10、8、7 cycles。总工作量给出 13 cycles 的下界，上式利用不可分割性给出 15 cycles。达到 15 仍可能受其他依赖和资源限制，这个例子只说明下界为何能加强。

图上递推 h_v, t_v 需要 $O(n+m)$ 时间；固定 a, b 后，检查集合成员需 $O(n)$ ，对选中操作排序需 $O(|S| \log |S|)$ 。枚举所有 a, b 还要另计成本。若 S 为空，不能仅凭 $a+b$ 得到有效下界。

B.3 计算工作量与不同核心之间的等待约束

考虑一串前后有依赖的特定计算操作。要求每个所选操作与其间全部相关计算都存在有向路径上的先后关系，从而将其他计算放入前后两个所选操作之间、互不重叠的时间区间。只有满足这一条件，后文才能把各区间的必要时间相加；任取几个汇合节点并不足够。

对已证明必须经过的分配到不同核心的计算操作之间的依赖，等待常数记为 τ_q ，问题一取 1000，问题二、三取 500 cycles。某个区间内选取 B.2 的非空使用同一种 Pipe 的操作集合，前后必要计算时长为 a, b 。令 $\ell, r \in \{0, 1\}$ 分别表示区间前后是否有所选约束操作。记 $(x)_+ = \max(x, 0)$ ，扣除前后计算已经覆盖的时间后，仍需考虑的等待为

$$c(a, b) = \ell(\tau_q - a)_+ + r(\tau_q - b)_+. \quad (\text{B.5})$$

设整个时间区间长度为 x ，可安排集合中操作的时间为 $z = x - a - b$ ，总计算工作量为 $W(S) = \sum_{v \in S} d_v$ 。若前后两个所选操作在同一核心，该核心最多可使用 z ，其他核心各至多可使用 $(z - c)_+$ ，故

$$W(S) \leq z + (K - 1)(z - c)_+. \quad (\text{B.6})$$

若前后操作位于不同核心，必有 $K \geq 2$ 。令 $A = (\tau_q - a)_+$ 、 $B = (\tau_q - b)_+$ ，可供同类计算使用的总周期数至多为

$$(z - A)_+ + (z - B)_+ + (K - 2)(z - A - B)_+. \quad (\text{B.7})$$

逐段比较 z 与 $A, B, A + B$ 可得

$$(z - A)_+ + (z - B)_+ \leq z + (z - A - B)_+, \quad (\text{B.8})$$

所以仍可采用前式限制工作量。对整数周期，满足 $W \leq z + (K - 1)(z - c)_+$ 所需的最小 z 为

$$\phi_{K,c}(W) = \min \left\{ W, \left\lceil \frac{W + (K - 1)c}{K} \right\rceil \right\}. \quad (\text{B.9})$$

当 $W \leq c$ 时， $z = W$ 满足第一段；当 $W > c$ 时，解第二段线性不等式并向上取整。结合操作不可分割的限制，得到

$$x \geq a + b + \max\{\phi_{K,c(a,b)}(W(S)), A_K(S)\}. \quad (\text{B.10})$$

满足开头条件时，不同区间及分隔计算互不重叠，可以把各自已证下界相加。如果只是从几个相互重叠的窗口得到下界，就只能取最大值，不能重复累计。问题一的 100 cycles 在同一个核心中切换 Task 的等待也不能自动加入，因为方案未必在所选计算处划分 Task。

由上述推导可见，与前后计算重叠的等待时间已被扣除，各时间段只计算一次。本节是补充理论推导，正文下界统计使用的是第 7 章所列已有版本的结果。

B.4 状态信息充分性的判定条件

本节讨论一个尚未由本文主算法一般实现的问题：能否只保存方案的一部分信息，就正确判断后续修改及其结果。两份方案当前 **Makespan** 相同，并不足以说明这一点。例如，它们可能采用不同的核心顺序，同一次移动操作在一份方案中合法，在另一份中却造成循环依赖。

先固定允许的修改集合 $\mathcal{A}$ ，例如在已经声明的结构范围内移动、合并或拆分子图；每种修改的参数含义也固定。用 D_a 表示允许执行修改 a 的方案集合，用 $\Phi(P)$ 表示需要保留的观测结果。若只研究成功执行，可取观测为 **Makespan**；若还包含失败，则须把成功或失败的状态纳入观测，不能把未定义的 **Makespan** 填成一个周期数。

令 $\mathcal{L}(P)$ 为从方案 P 出发能够连续执行的所有有限修改序列，包含不作修改的空序列。两份方案在本节所关心的意义下等价，要求

$$\mathcal{L}(P) = \mathcal{L}(Q), \quad \Phi(w(P)) = \Phi(w(Q)), \quad \forall w \in \mathcal{L}(P). \quad (\text{B.11})$$

其中 $w(P)$ 表示按序列 w 依次修改 P 。第一个条件保证可做的修改完全一致，第二个保证修改后的观测一致；只比较双方碰巧都能做的修改会漏掉第一类差别。

命题 B.1. 设 $S(P)$ 为准备保存的信息。对每种修改 a ，若存在判定函数 d_a 、更新函数 $\bar{a}$ 和观测恢复函数 o ，使声明范围内的全部方案满足

$$P \in D_a \iff d_a(S(P)), \quad S(a(P)) = \bar{a}(S(P)), \quad \Phi(P) = o(S(P)), \quad (\text{B.12})$$

且更新等式在 a 可执行时成立，则 $S(P) = S(Q)$ 蕴含上述等价关系。

证明. 不作修改时，由观测恢复等式知两份观测相同。若执行相同的前 j 步后保留的信息仍相同，则 d_a 保证下一步在两份方案中同时允许或同时不允许；允许时， $\bar{a}$ 使更新后的信息仍相同， o 再恢复相同观测。对序列长度归纳，即得全部可执行序列与后续观测一致。□

命题给出充分条件，没有证明保存的信息已经最少。要说明最少，还需证明不同保存状态必能被某个后续修改或观测区分。把整份方案编码成一个整数没有减少信息量；重复使用相同编译结果、保存程序运行现场、删除完全相同的候选，则分别解决较小的问题，不能自动获得本命题关于所有后续修改的保证。

附录 C 固定实现与复现信息

三题固定求解器的提交分别为下列 SHA。逐输入的图、方案与结果来源及完整 SHA-256 见配套 CSV 与来源收据。固定版本结果不能用不同算法的历史赢家拼接。

- P1: 834d8c957538ee069c66aadac9509552a4cc69d7; 求解过程中的评分至多 10 次 E1。
- P2: c66559a6f8a31ef7b4720e1f7c3c28d61f8dff3f; 求解过程中的评分至多 3 次 E2。
- P3: 311322b996c0948e8a6a9c7ec6ddfe6ae41fbee1; 求解过程中的评分至多 3 次 E0。

官方代码与配置身份分别为 de11a83db8d7c47ed328b15a7df71d613a833b16cd23ee9fe877999578a1ace0 和 dcd10de54b23f8366428fb24e828812b1da9549e6eae4a3c3f38604fe5ae77b9。来源收据记录 822 个固定 Git 对象；本稿的数据导出仅读取已有原件，没有新增求解器或评价器调用。